

GPU Programing



Contents

01

Computer hardware and CPU
Programing

02

GPU Architecture

03

GPU Software

04

GPU Programming principles
with Triton

05

Torch Kernel Ecosystem

05

Conclusion

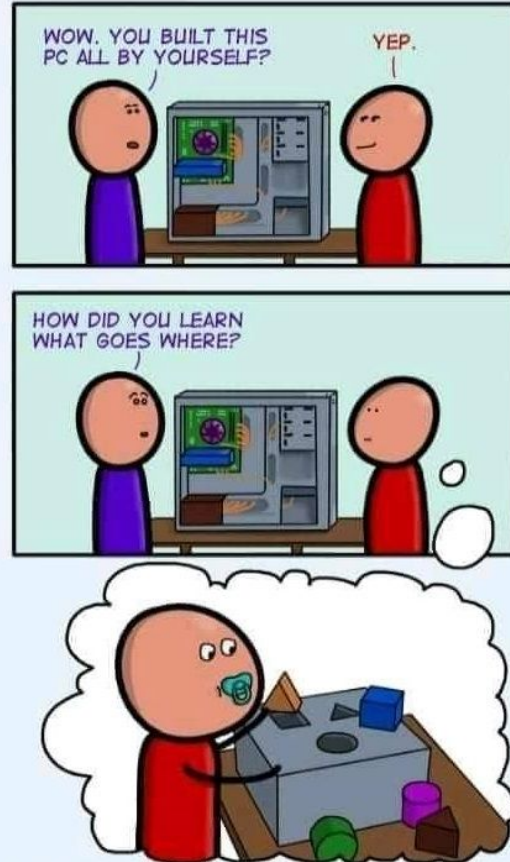
A decorative header with a blue geometric pattern of overlapping triangles and lines.

Computer hardware and CPU Programming

First time for everyone

Have you ever build your first PC by picking every pieces of hardware by your own ?

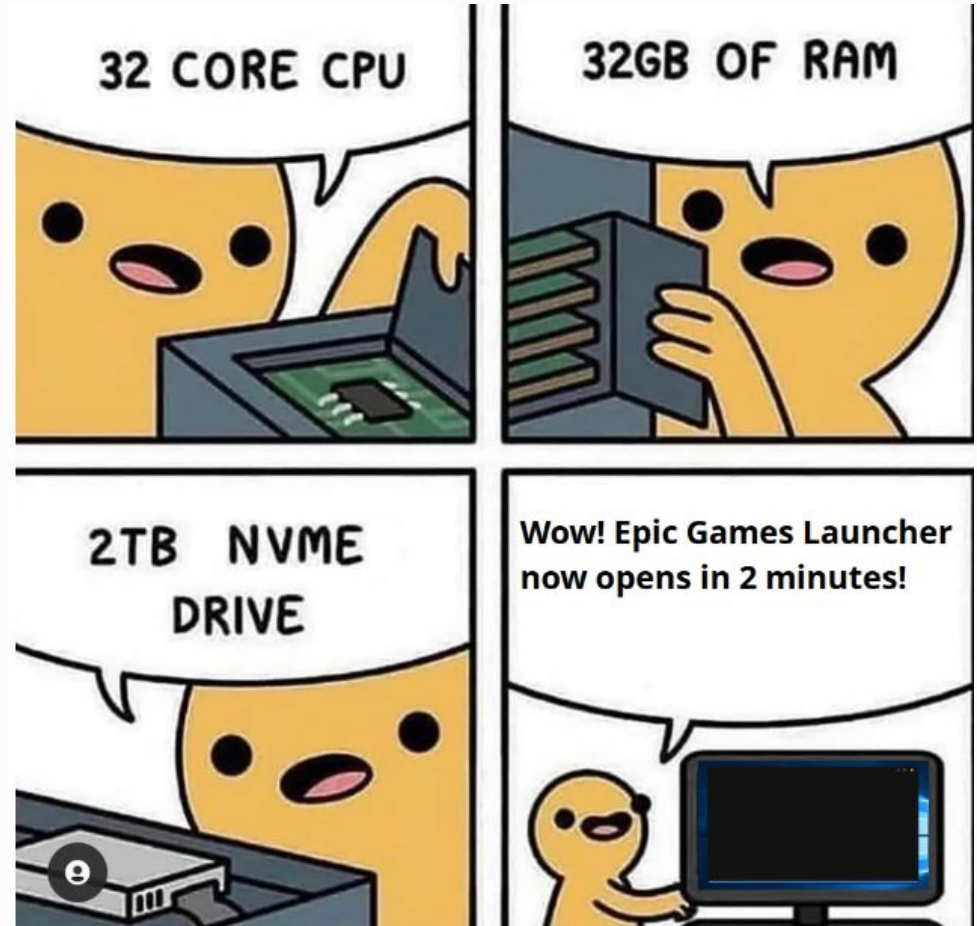
First time for everyone



This probably you (and me) when I build my PC for the first time, there is a bunch of technical term and hardware that I need to research how it works.

First time for everyone

You open some youtube tutorials about PC components and pick every parts by your hand without know nothing about how it work.



First time for everyone

After researching core components of a PC (probably for gaming), you assume that:

- A giga chad GPU for gaming to achieve 4k resolution, shader, motion, light, bla bla
- A water cooler that make you cooler
- More RAM for a bunch of Chrome tabs that you never close
- Picking Intel I5 always better than I3 no matter what
- SSD for fast boot the system up
- A large mainboard with bunch of connection ports just for nothing

A Tryhard's "First PC Build"



only 8600K or 2600X



ALWAYS watercooled for some reason



RGB RAM!!!



mobo: usually Asus, always full ATX even though they only have 1 PCIe card, also must have a plastic IO cover



overpriced NZXT case



GTX 1080Ti



overkill Corsair PSU



always a 1TB HDD and 250-500 GB SSD



First time for everyone

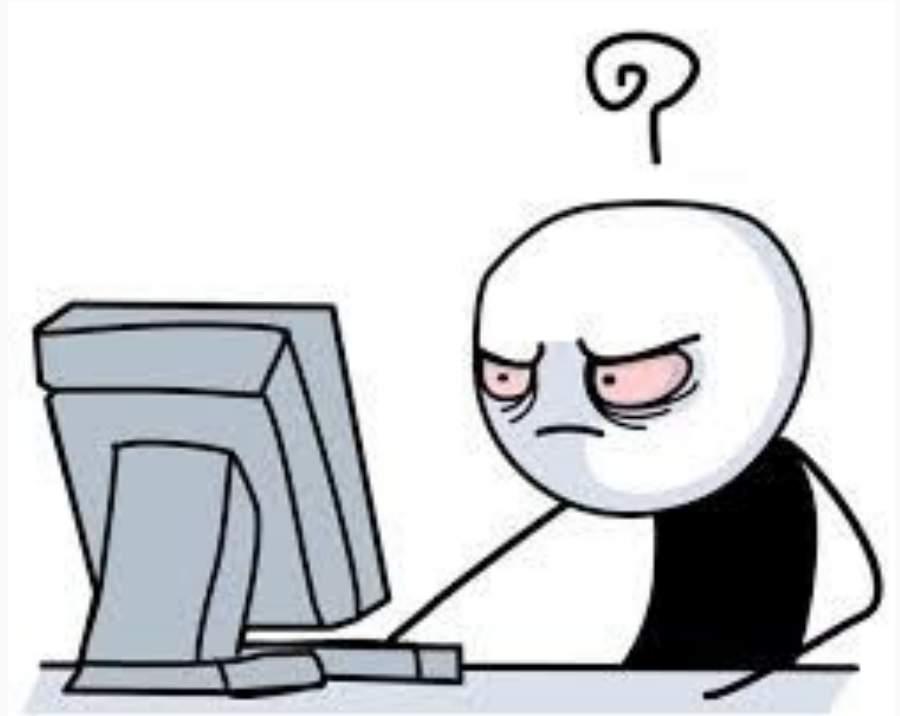
When you turn on the pc
you built for the first time



And after buying these hardwares and
plug it together, you wait for the moment of
truth.

First time for everyone

Luckily, you are an undergrad student, majoring in Computer Science with curiosity mind and start wondering how the f**k do my computer work

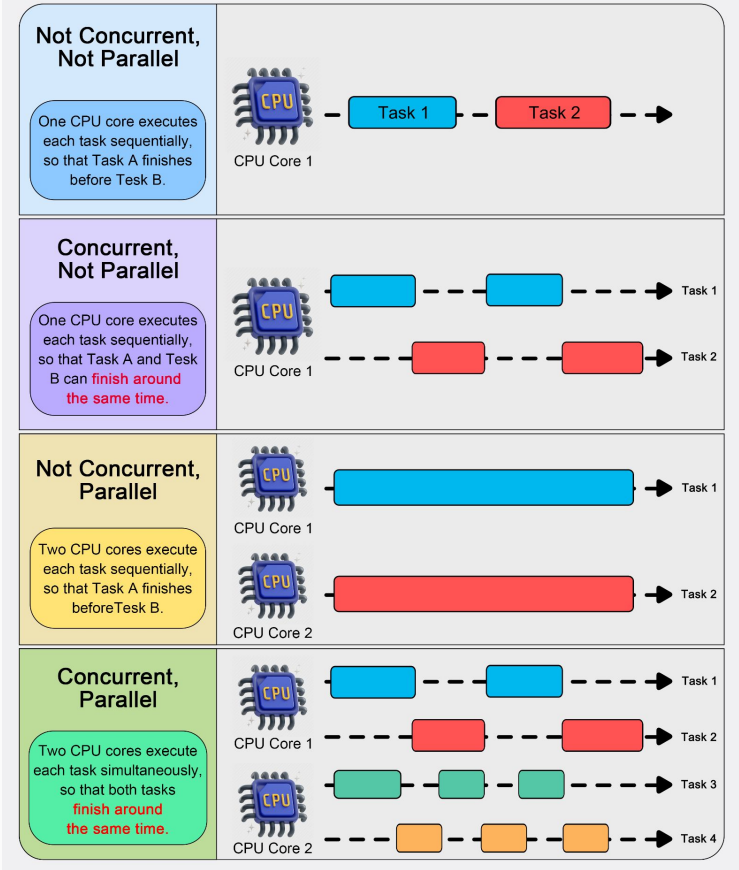


Concurrency vs parallelism

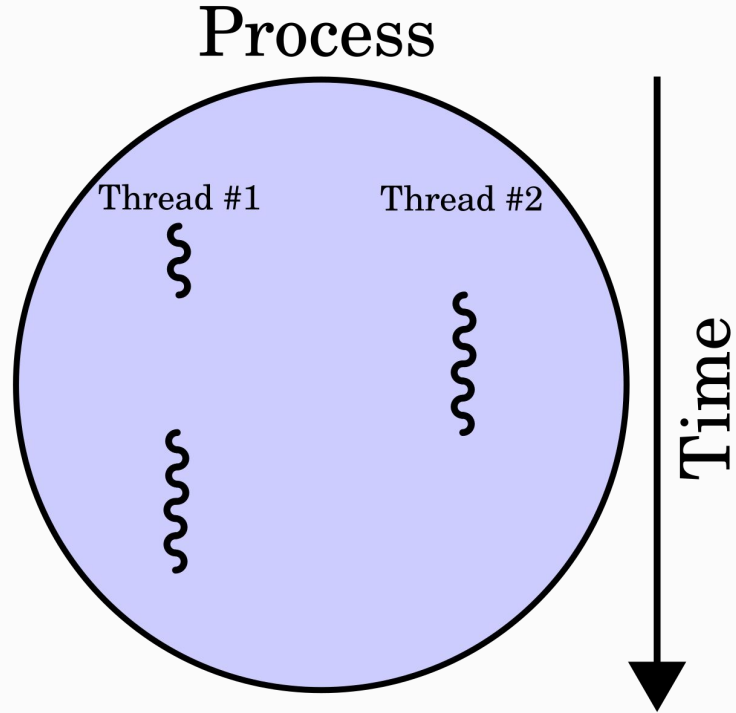
Concurrency hides idle time by overlapping tasks, while parallelism uses multiple cores to run tasks simultaneously.

Concurrency is **Not** Parallelism

ByteByteGo



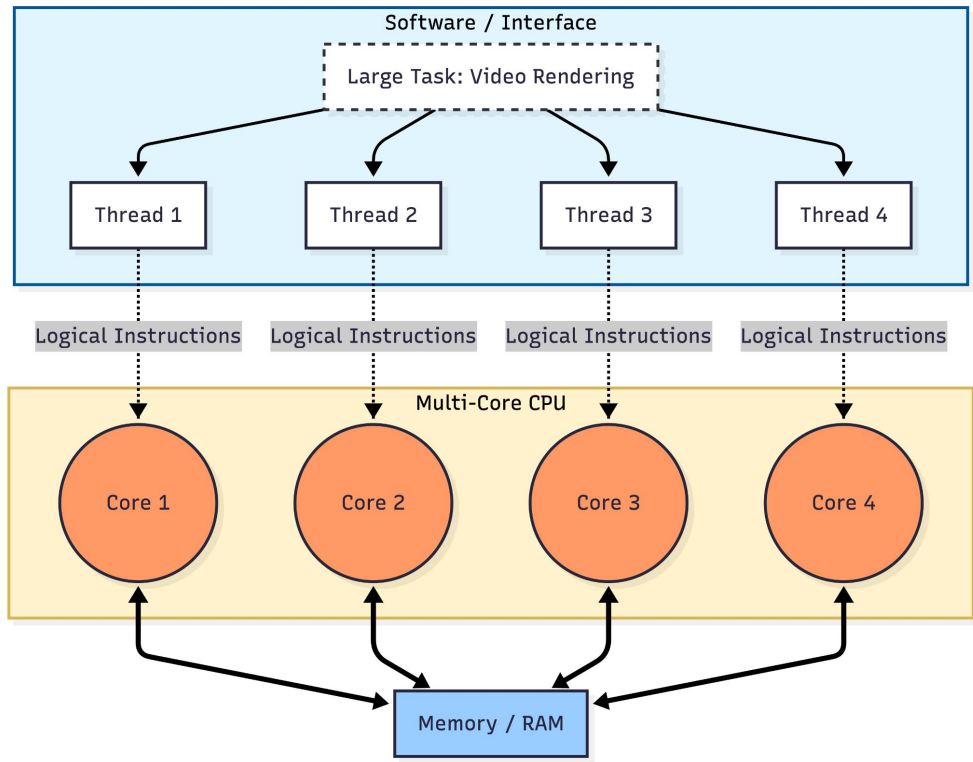
Thread



A task or process consists multiple threads
each thread is a logical sequential
execution.

When the computer run a process,
multi-thread

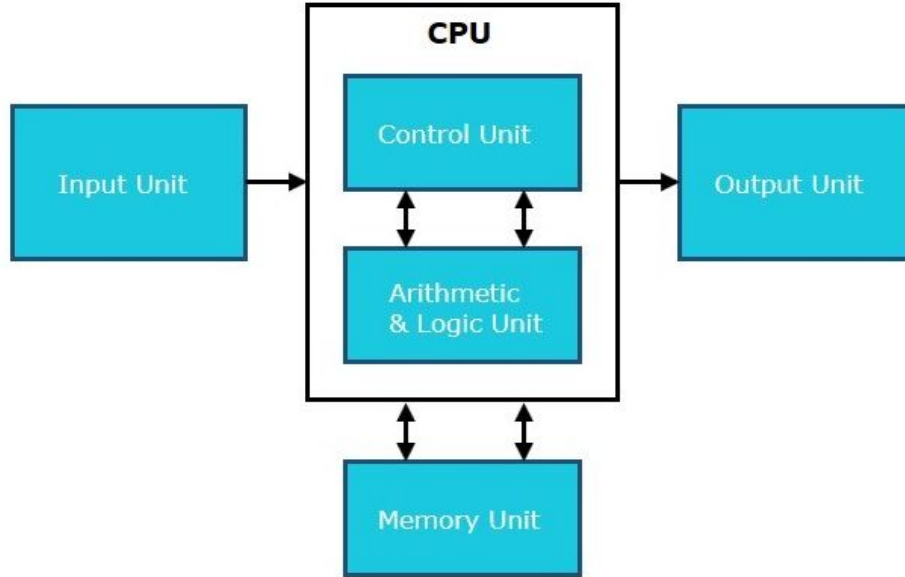
Multi-threading



You have a large video rendering task which split into 4 threads, assume each thread is assign to 1 core only

This high level diagram show how your task script is execute in CPU

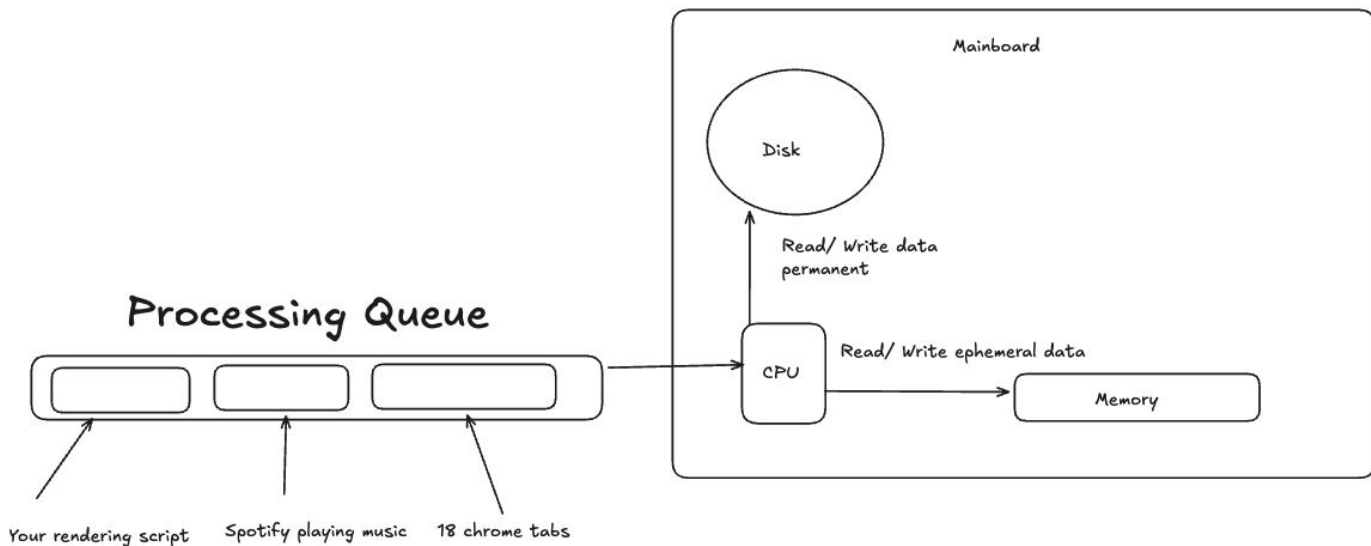
Why traditional computation is so slow



Dive deeper, you know that the CPU constantly fetch data from RAM and make calculation step with the Arithmetic Logic Unit (ALU) and transfer data back and forth with Control Unit

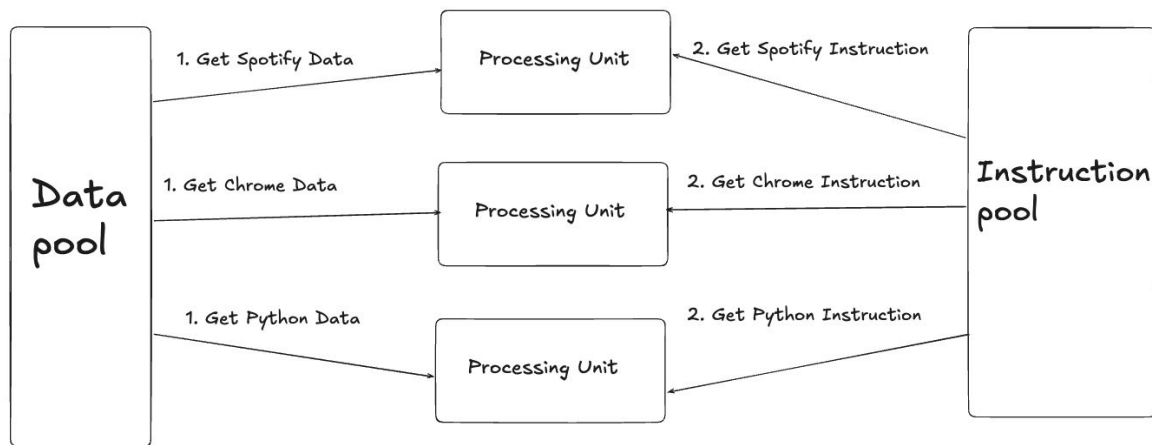
Why traditional computation is so slow

But the fact that the CPU is not handling a single script task, it also handling other tasks in the processing queue like 18 chrome tabs, your spotify playing music, etc... So CPU ALU is designed to be all rounded swiss army knife to handle all those tasks



CPU Execution Model

CPU Multiple Instruction Multiple Data (MIMD) Execution model



Follow Michael J. Flynn's execution model. This is a conceptual taxonomy on how CPU work

Map to previous example where you have 3 instructions for 3 tasks, each task have different data to be execute on CPU.

Why GPU ?



**NOOO! YOU CAN'T
BRUTEFORCE PW
ANYMORE IT'S OBSOLETE**

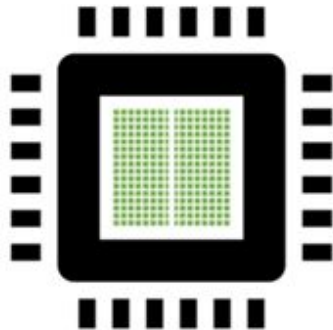
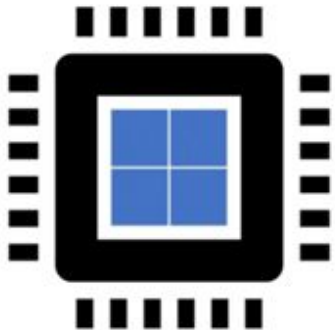
imgflip.com



**HAHA
GPU GO BRRR**

You found that a random guy on Youtube said you should use GPU for your rendering script and it actually go faster

Why GPU ?

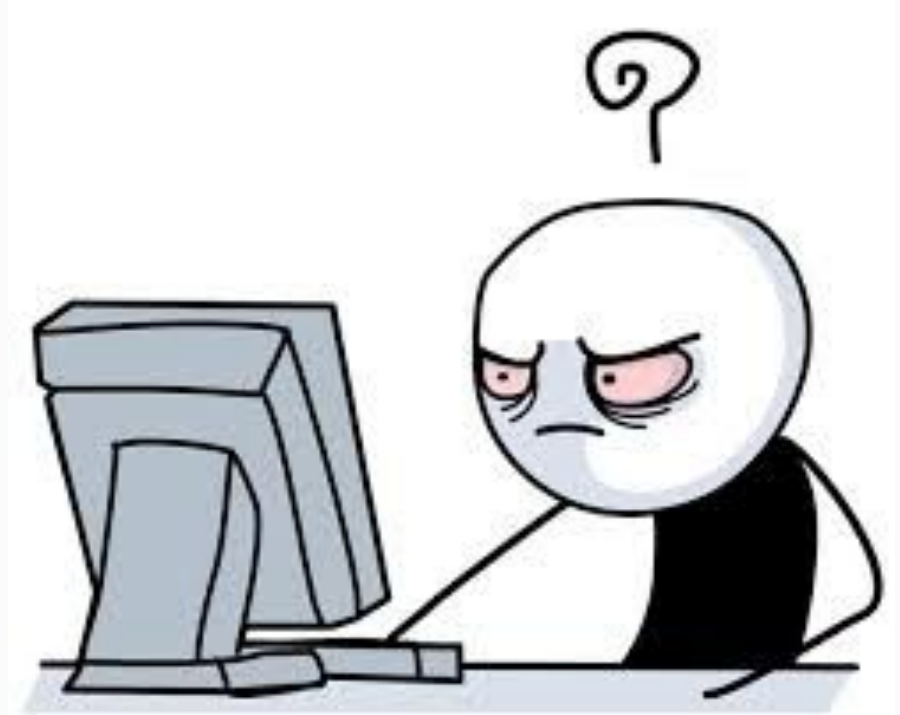


This is a core comparison on how GPU is selected for scientific and parallel programming

CPU	GPU
Central Processing Unit	Graphics Processing Unit
4-8 Cores	100s or 1000s of Cores
Low Latency	High Throughput
Good for Serial Processing	Good for Parallel Processing
Quickly Process Tasks That Require Interactivity	Breaks Jobs Into Separate Tasks To Process Simultaneously
Traditional Programming Are Written For CPU Sequential Execution	Requires Additional Software To Convert CPU Functions to GPU Functions for Parallel Execution

Why GPU ?

And you staring your screen again and wonder how GPU really go brrrr with your rendering task.



Disclaimer

There are many factors affect on GPU performance compare with traditional CPU especially in scientific computing likes.

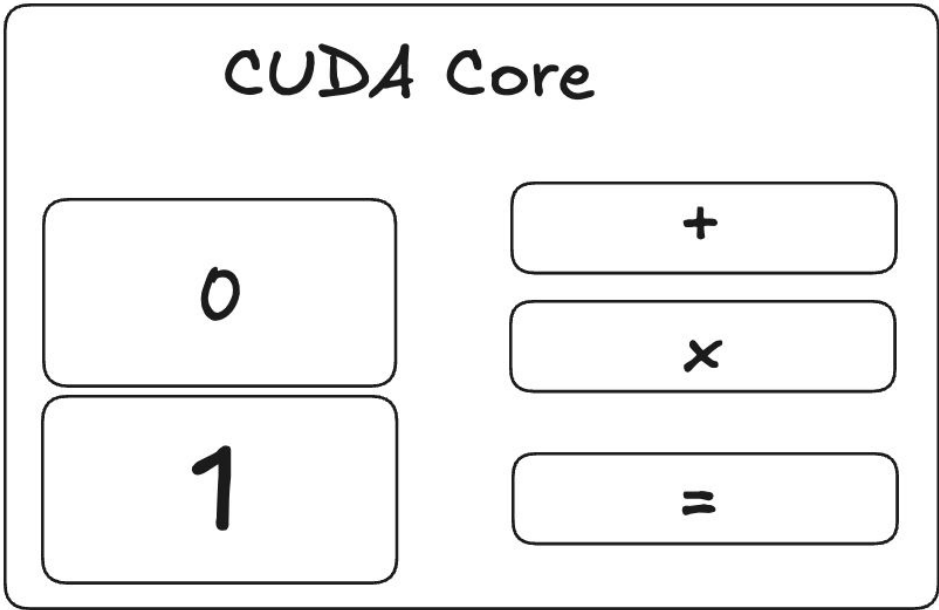
- Data Transfer: The PCIe & RAM Bottleneck
- GPU Interconnection between nodes, clusters
- Thermal limit from GPU and other devices
- How precise do you want your GPU to perform
- GPU firm and generation
- The scale of your input data
- ...

In this presentation, I'm just focus to the core idea of GPU architecture, why GPU is use for high performance computing and how you can modify your own kernel.

The top of the slide features a decorative header with various shades of blue and white geometric shapes, including triangles and lines.

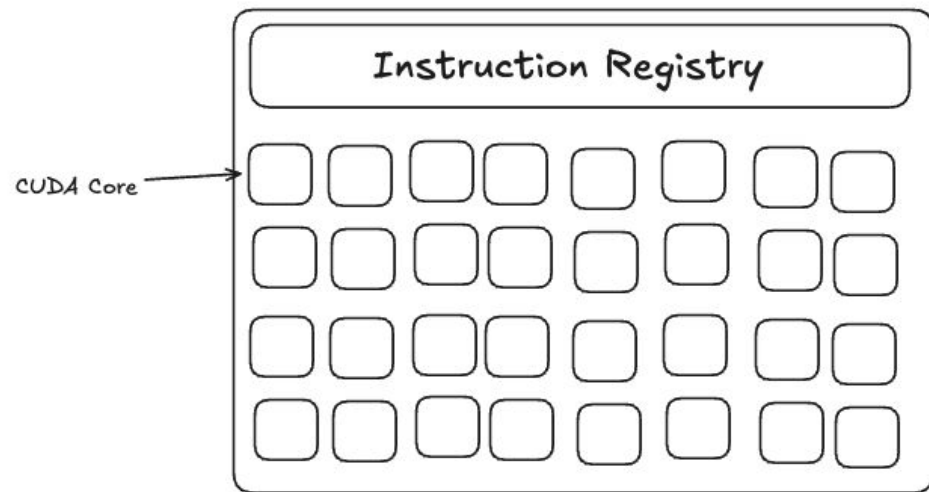
GPU Physical Architecture

GPU Physical Core Component



CUDA core is the worker unit in a GPU, it only capable of doing simple task like add and multiply. Imagine it like a calculator with 2 function and 2 number 0 and 1 only

GPU Physical Core Component

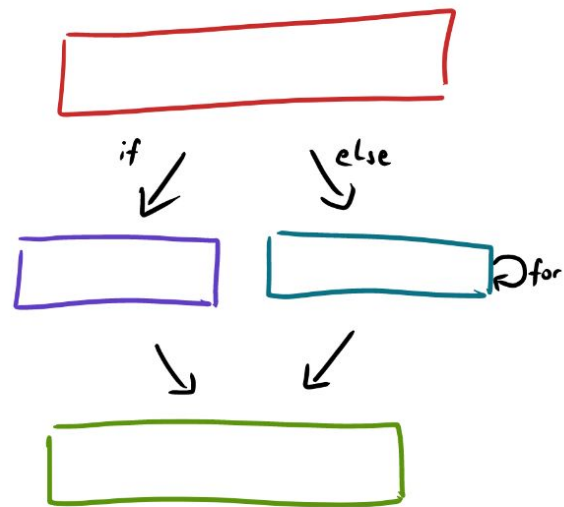


A warp is the smallest scheduling unit in a GPU. It consists of 32 threads that share a single instruction stream. These threads are executed by CUDA cores over one or more cycles, depending on hardware availability. CUDA cores are shared across many warps and are not permanently assigned to any single warp.

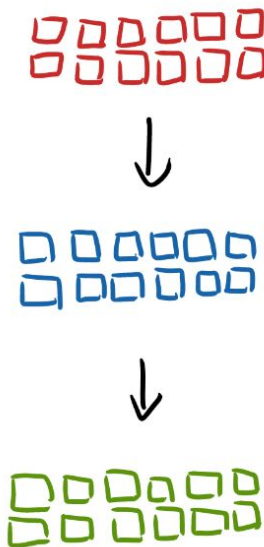
The Instruction Register functions like a brain which orchestrates and instructs all CUDA cores to compute by task.

High level GPU architecture

CPU:



GPU:



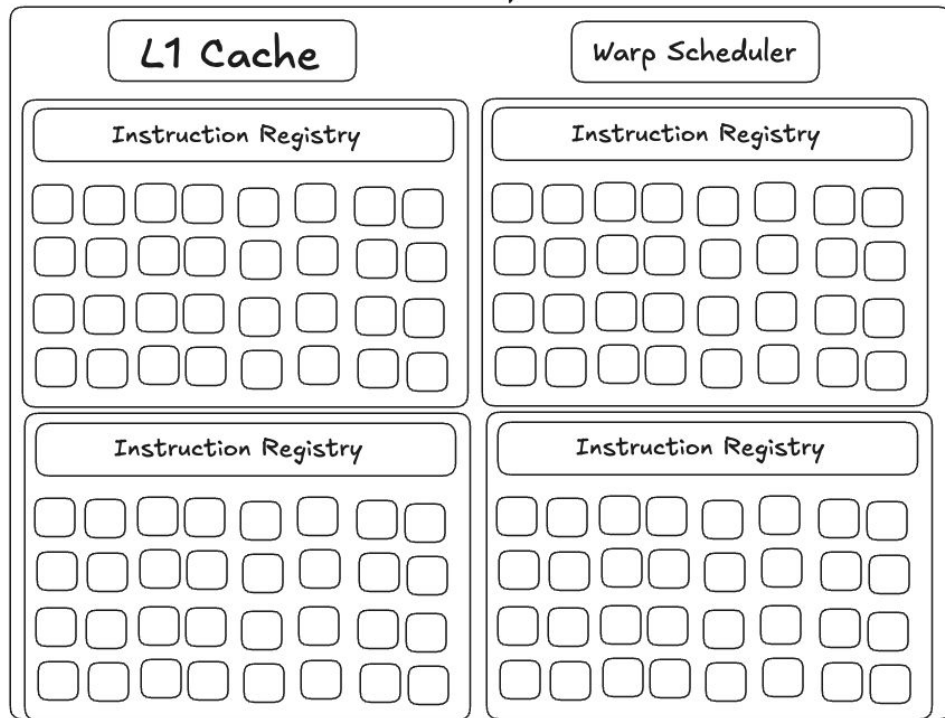
This illustrates a key architectural difference between CPUs and GPUs.

CPU cores are built for complex control flow, using sophisticated logic to handle branching and sequential execution, with each core acting as an independent worker. GPUs, by contrast, execute threads in warps, where many lightweight CUDA cores perform the same instruction simultaneously with simple control logic.

As a result, GPUs are well suited for scientific and high-throughput workloads that apply the same computation across large datasets.

High level GPU architecture

Streaming Multi Processor

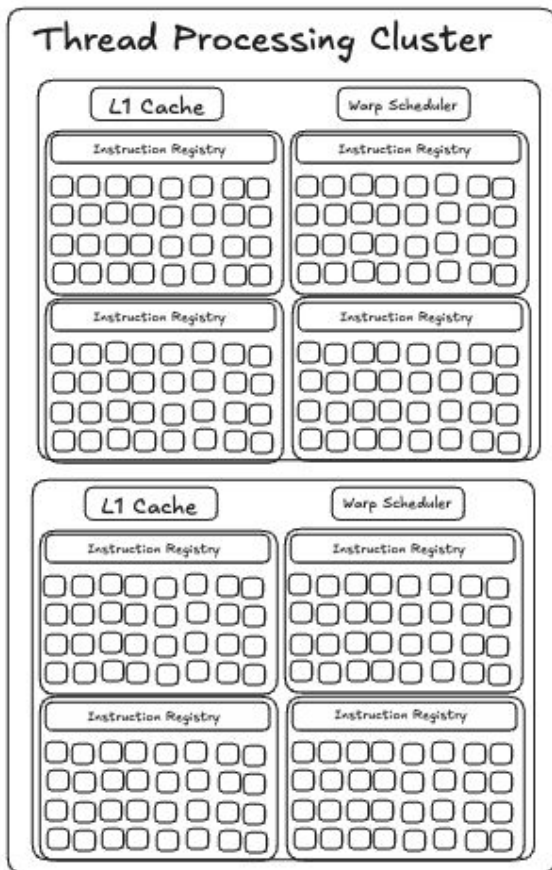


The lowest compute unit is Streaming Multiprocessor consists up to 4 Warps. Each SM has private registers and L1/shared memory for low-latency execution

In reality, the warps are queue up to 64 (depends on generations) by warps scheduler.

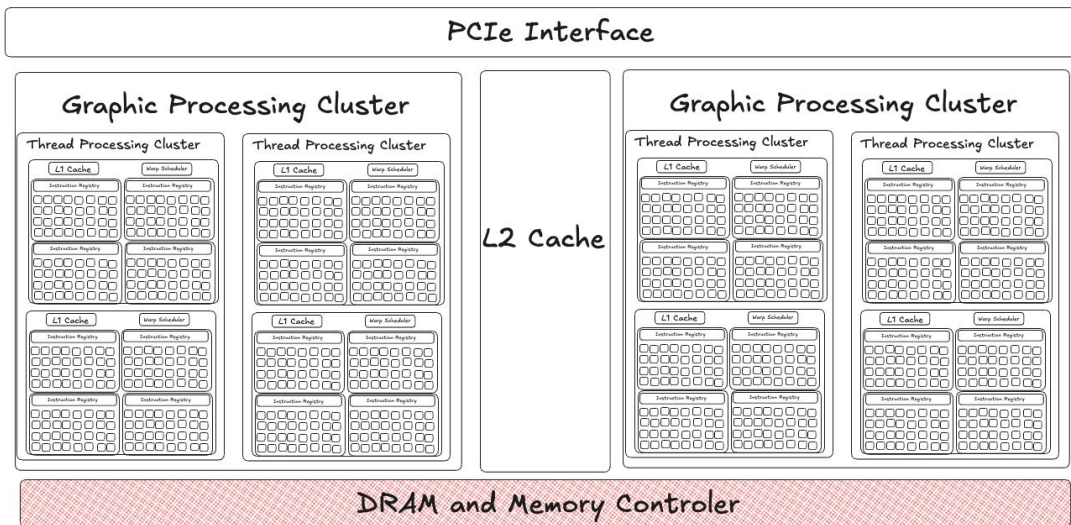
The SM is take charge for each pieces of instruction break in kernel tasks.

High level GPU architecture



Two consecutive SMs are grouped together to form a Texture Processing Cluster (TPC). These TPCs are supported by an additional infrastructure layer that includes L2 cache partitions and memory controllers, which are responsible for fetching data from VRAM.

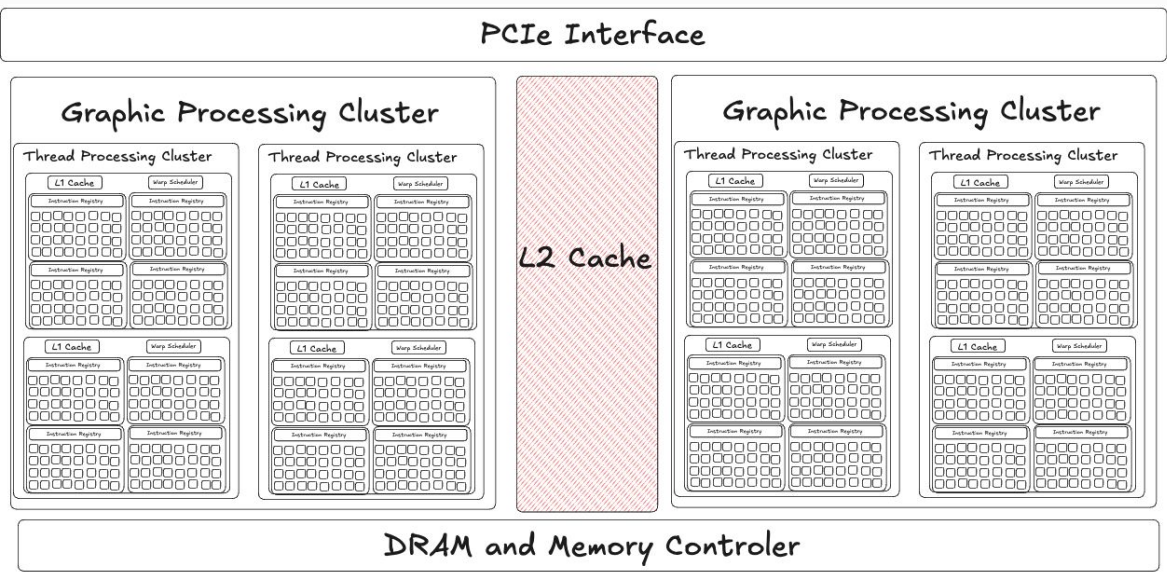
High level GPU architecture



This is the most well-known component, commonly referred to as VRAM or GPU DRAM. Consumer GPUs typically use GDDR memory, while data center GPUs use HBM.

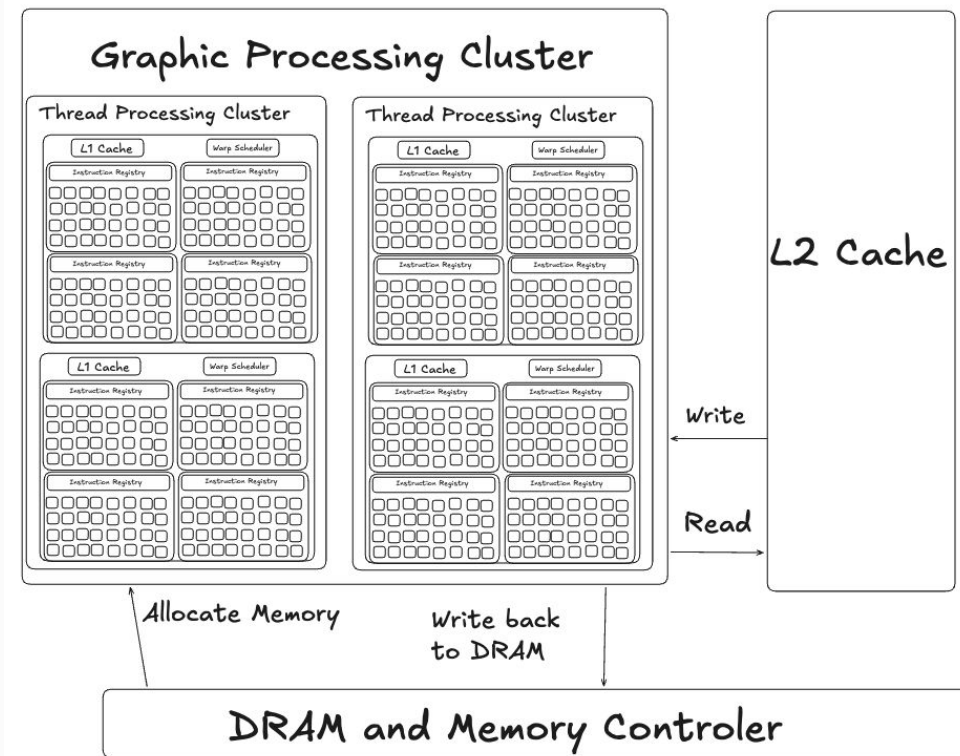
Its role is similar to system RAM in a computer; however, the key differences are that it is integrated directly into the GPU rather than installed in a motherboard slot, and it is specifically optimized for extremely high data-transfer bandwidth.

High level GPU architecture



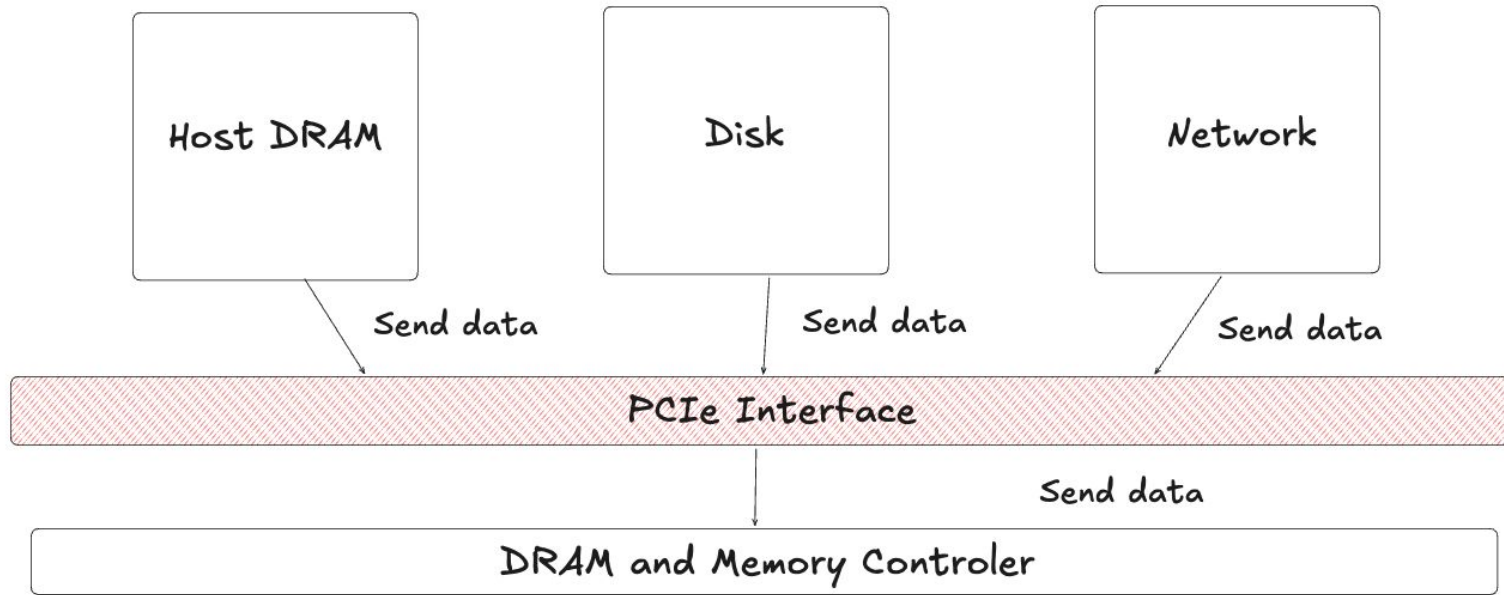
All GPC shared a same L2 Cache for low latency data retrieval. The latency is much lower than DRAM but higher than L1 cache inside each TPC

High level GPU architecture



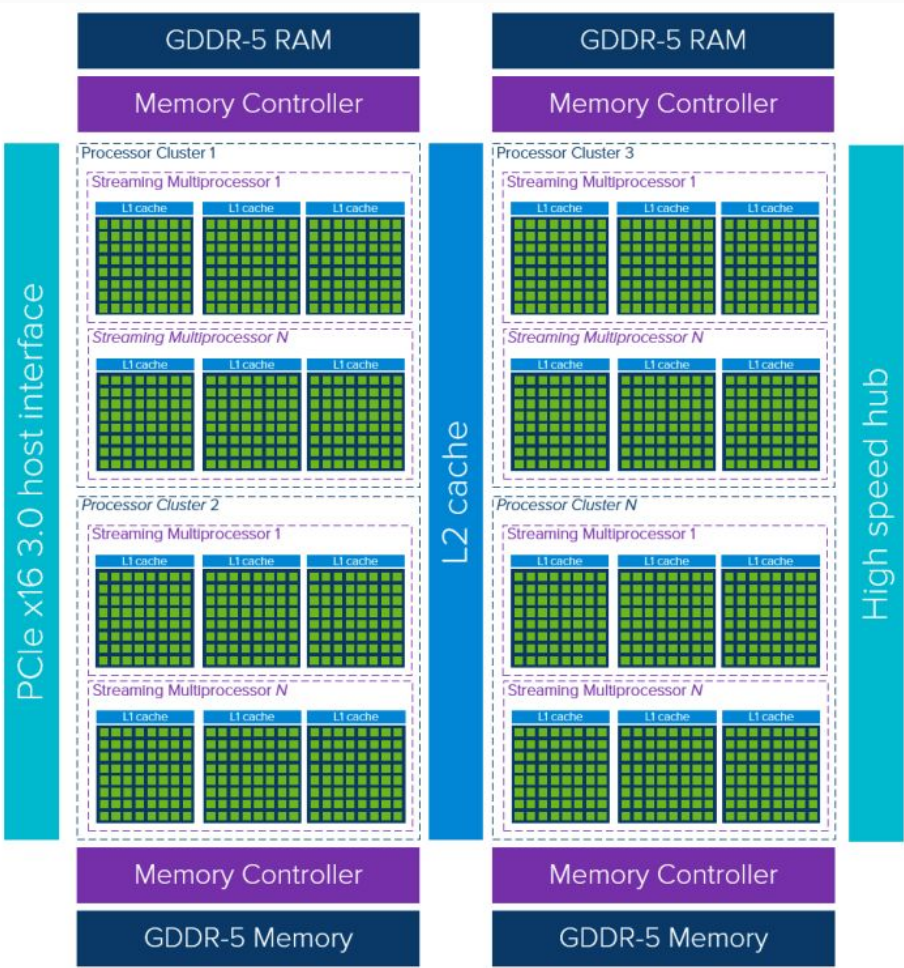
A simplified workflow how an GPC, L2 cache with global memory module work together.

High level GPU architecture



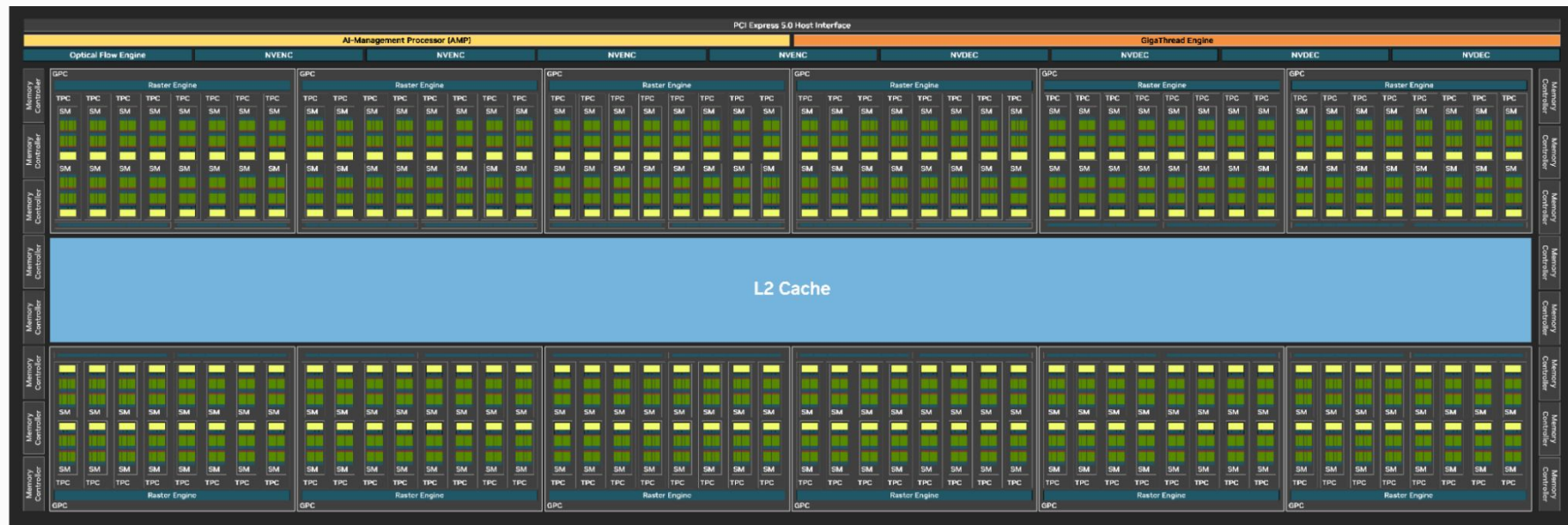
The PCIe Interface is an universal standard for every hardware module to send the data from external sources to GPU

High level GPU architecture



After discussing all the components in the previous slide and how they operate, this diagram presents a simplified view of GPU architecture. All of the compute units mentioned earlier are constructed from billions of transistors working together.

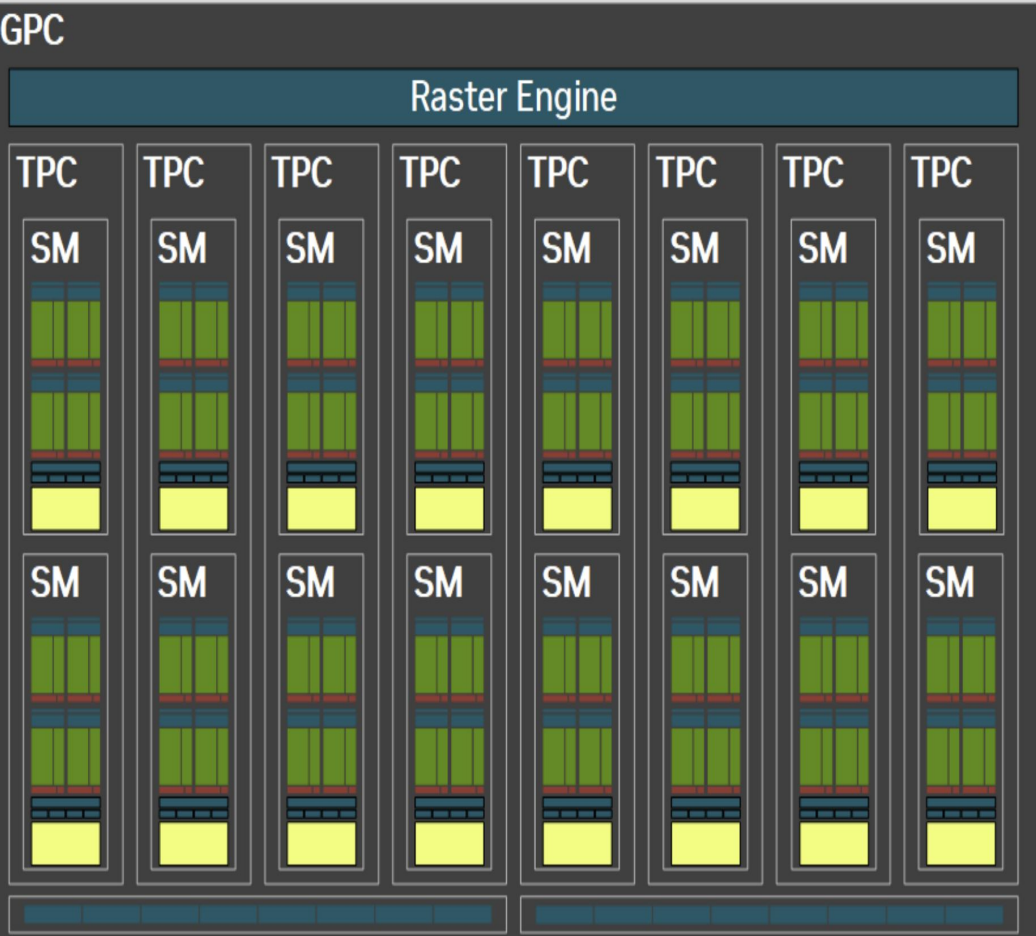
Introduction to NVIDIA Blackwell GPU chip



The full GB202 chip install on a RTX 5090 consists of 92.2 billion transistors include:

- 24576 CUDA Cores
- 192 RT Cores
- 768 Tensor Cores

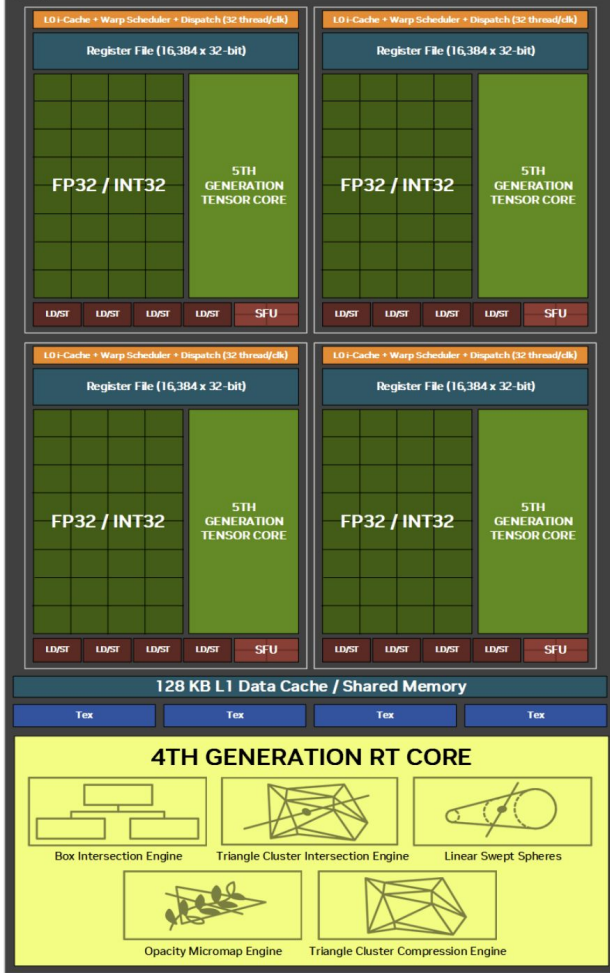
Introduction to NVIDIA Blackwell GPU chip



The Blackwell GB202 chip has total 8 TPCs for a single GPC. It also include with a Raster Engine (for image tasks only).

Introduction to NVIDIA Blackwell GPU chip

SM



There are 144 SMs in Blackwell

- Each SM contains:
 - 128 CUDA Cores
 - One 3rd-generation RT Core
 - Four 4th-generation Tensor Core
 - 256 KB Register File
 - 128 KB L1 Cache

Introduction to NVIDIA Blackwell GPU chip

Blackwell SM

LO i-Cache + Warp Scheduler + Dispatch (32 thread/clock)

Register File (16,384 x 32-bit)

FP32 / INT32

5th Generation
Tensor Core

LD/ST

LD/ST

LD/ST

LD/ST

SFU

Each SMs consists of CUDA cores for simple arithmetic computation and 1 Tensor core for Tensor computation

The LD (Load) and ST (Store) unit is used for data transferring

The SFU (Special Function Unit) pre-define some complex arithmetic function for CUDA core to compute approximate like: exponent, root, ...

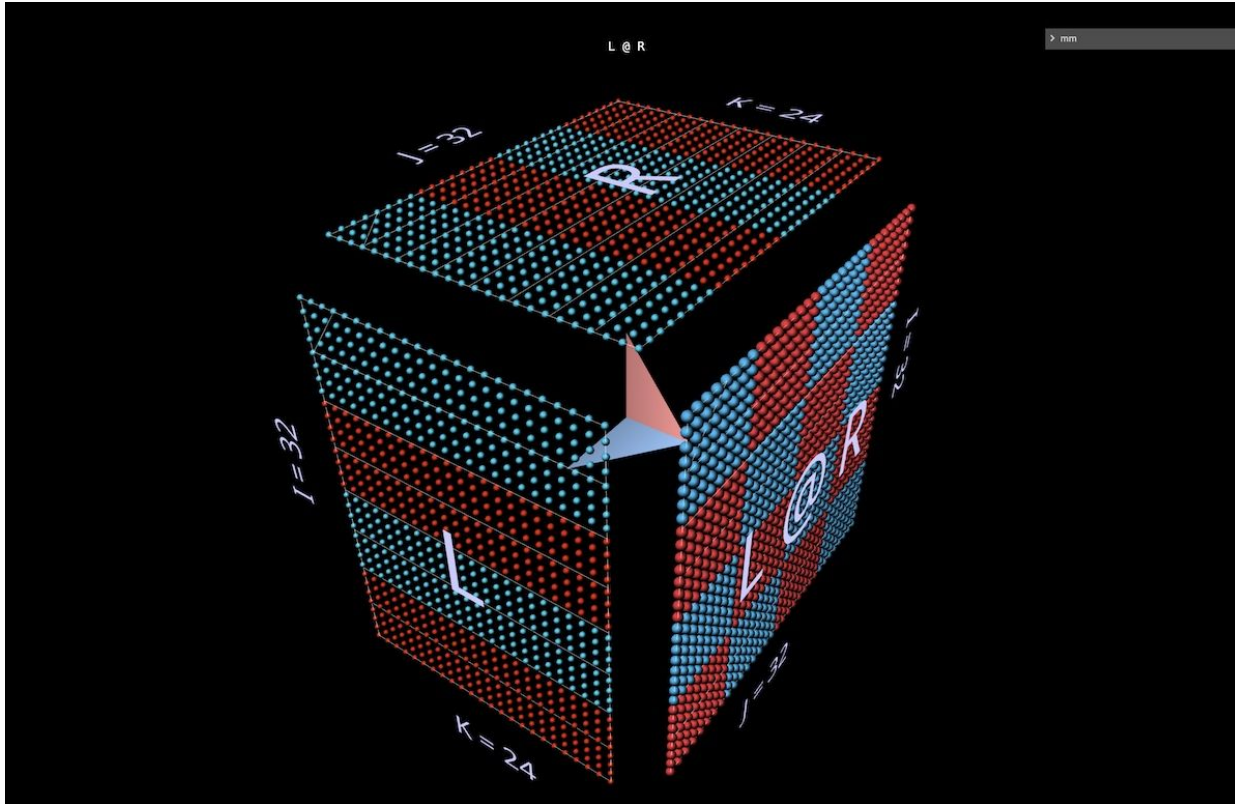
The top of the slide features a decorative header with various shades of blue and white geometric shapes, including triangles and lines.

GPU Logical Architecture

CUDA Kernel

A kernel is the unit of CUDA code that programmers typically write and compose, akin to a procedure or function in languages targeting CPUs.

CUDA Kernel



A simple matrix multiplication can be a CUDA kernel how GPU calculate this function.

A Kernel launch a grid in logical execution. Grid is the highest logical compute unit

CUDA Thread



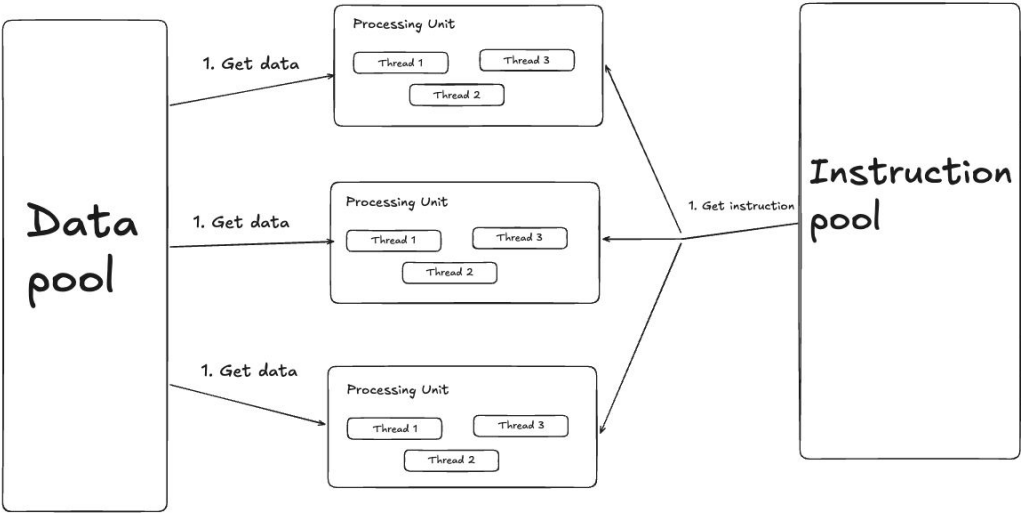
Thread is the lowest logical execution unit it define each computation step.

You can think it like a recipe in your mind how i should cook my dish

But this recipe is constraint by the physical chef which called Warp Scheduler what, how and where the dish is cook

GPU SIMT execution model

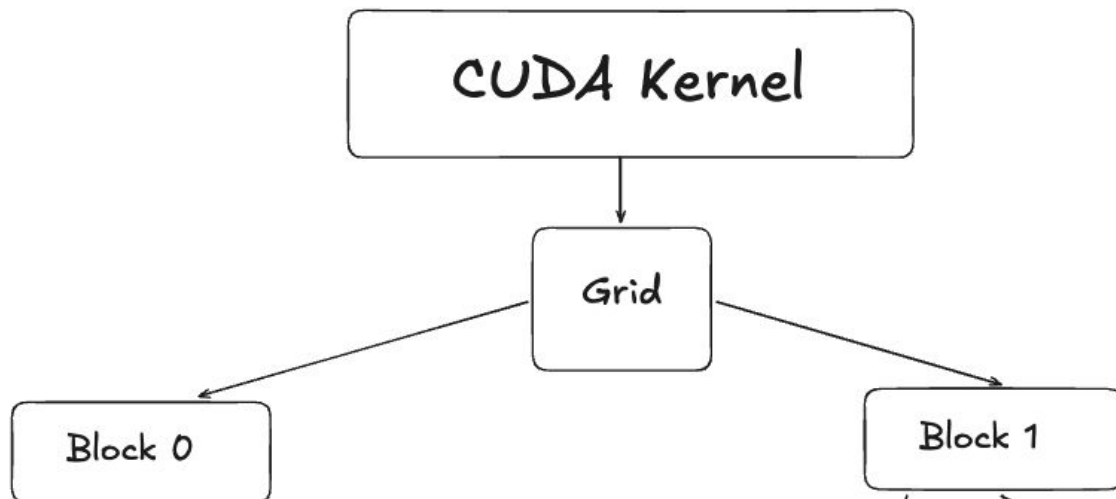
GPU Multiple Instruction Multiple Threads (SIMT) Execution model



Before moving on how GPU achieve high throughput, modern GPU adopt the Flynn's execution taxonomy

A GPU executes a single instruction per warp, which is applied simultaneously to multiple threads operating on different data elements under the SIMT execution model.

GPU logical execution

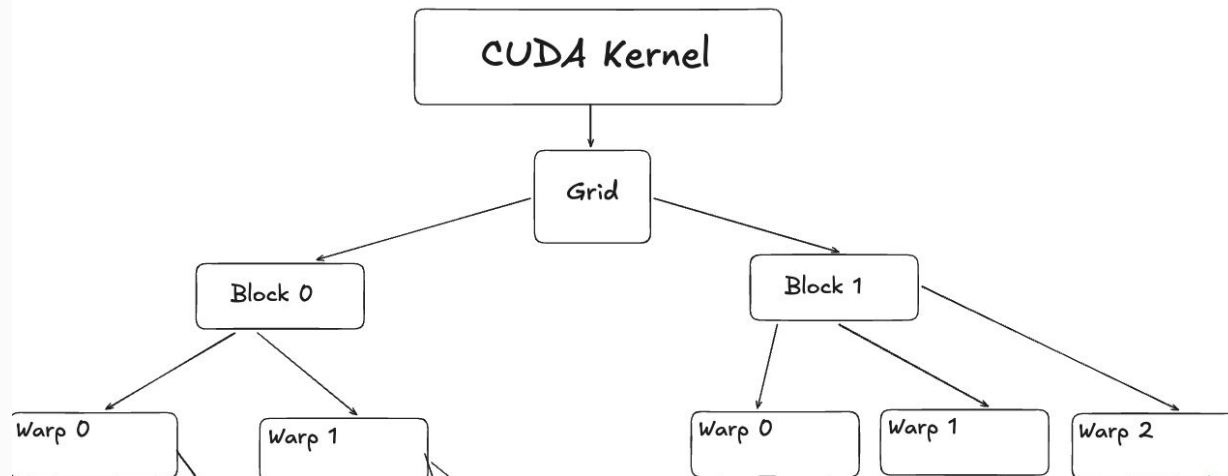


A Grid split the computation into block, each block represent a similar computation portion of a grid

For example, think a CUDA Kernel is like a boss order a huge group of worker in a Grid.

The head worker decide to split the workforce into 2 group Block 0 and 1. These Block doing task independent from each other

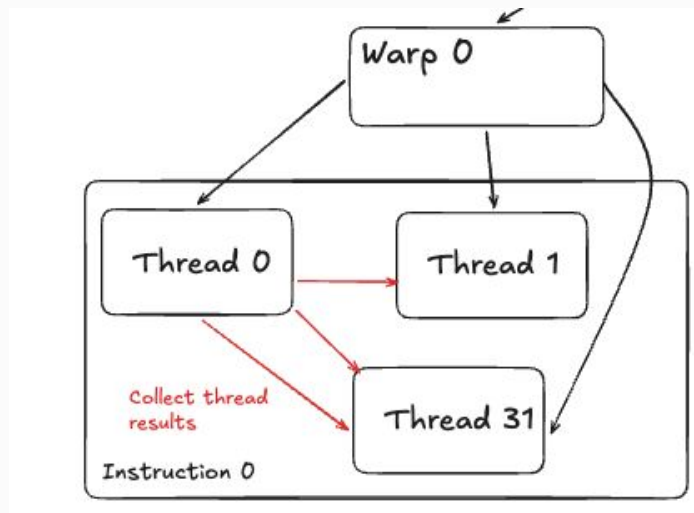
GPU logical execution



Warp is a smaller unit divide by a block for another computation part.

In our example, after split into Block, now the worker decide a smaller group called Warp to do their part of work separately. Due to Block 1 task is heavier but the number of people is same as Block 0, they decide to split into 3 Warps.

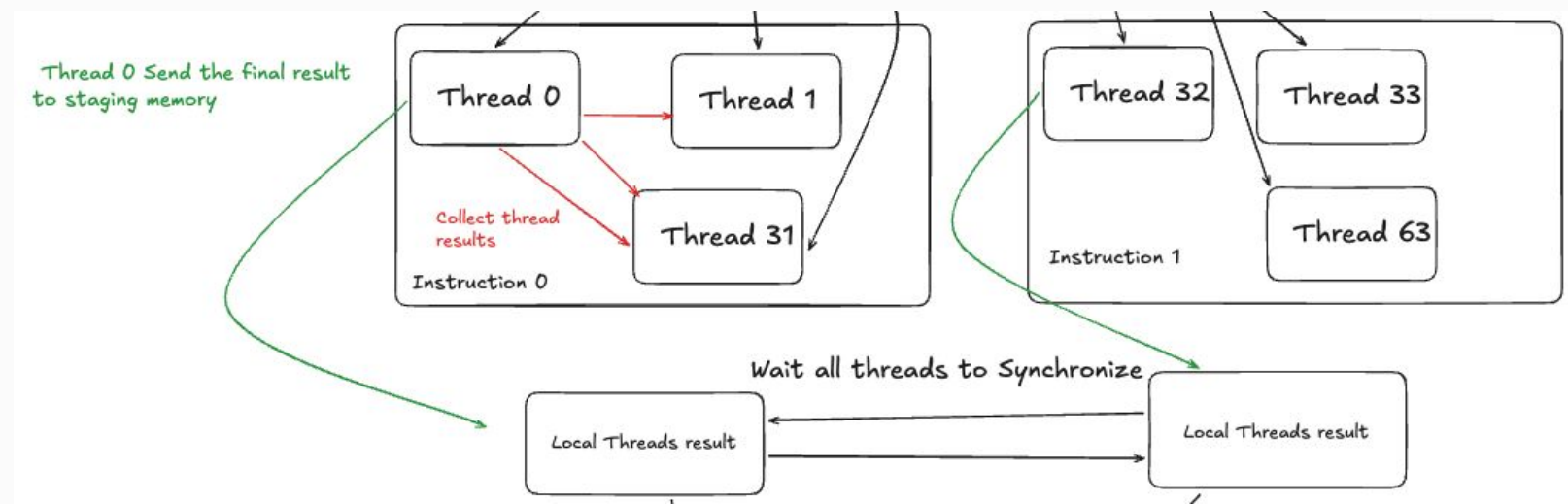
GPU logical execution



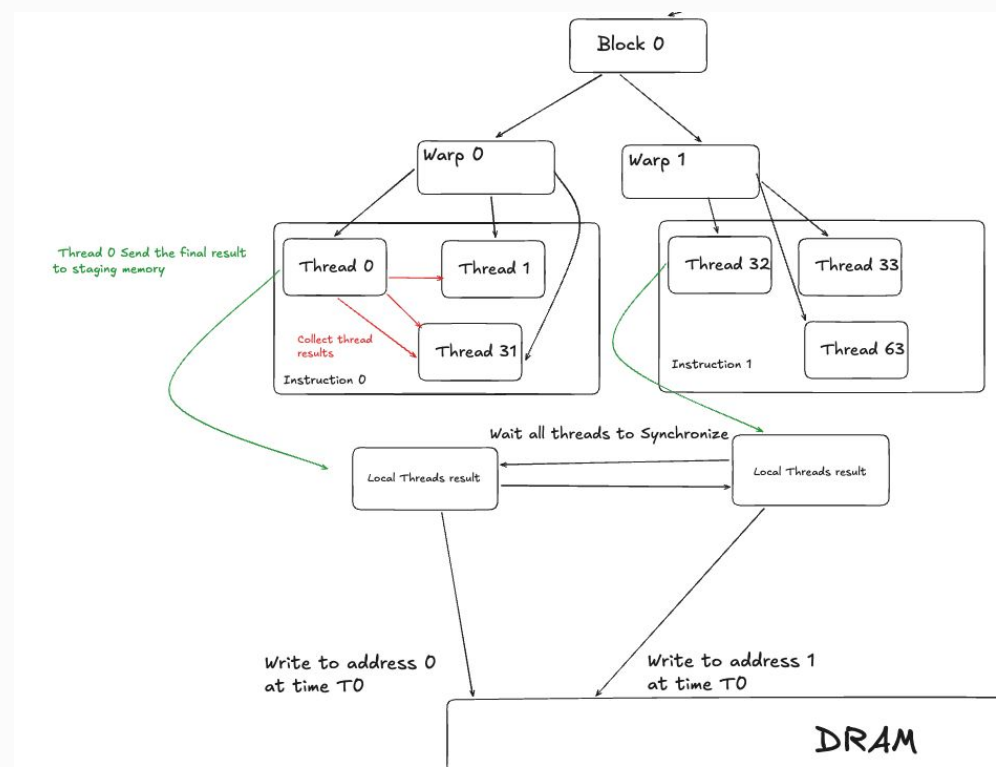
Each of the Warp group has total 32 individual workers called thread. These needy people work is assign to specific job, after their work is complete, the Thread 0 act as a leader to collect all result before talk to other leaders

GPU logical execution

But there are 2 sub tasks for the Block 0 and not all of worker perform equally . So the leader Thread 32 and Thread 0 decide to wait for each other before report the result

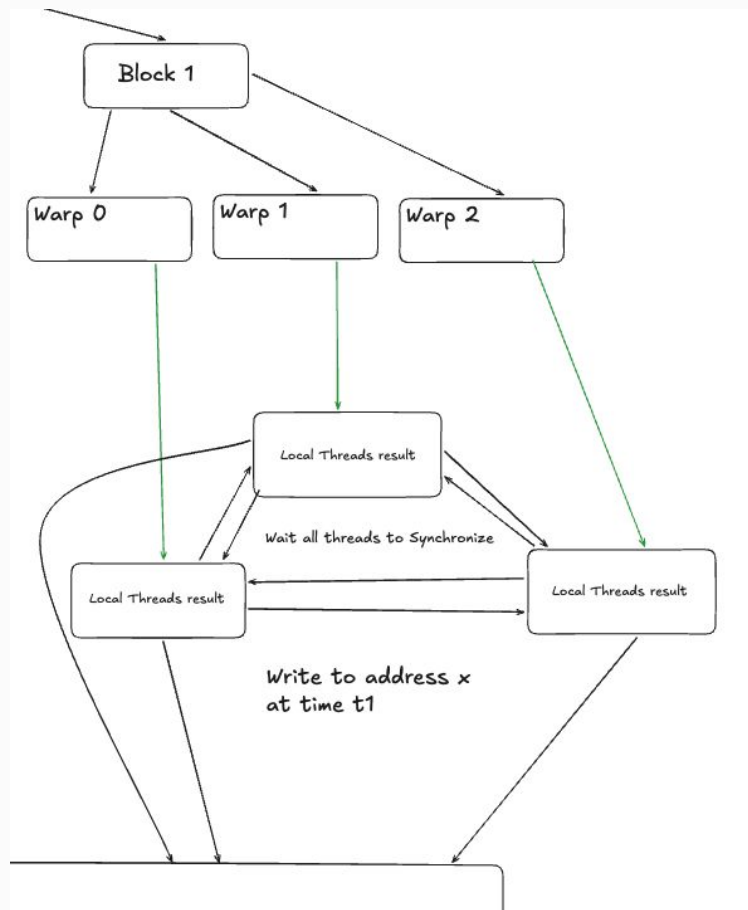


GPU logical execution



After both leader collect all worker result, they both log their team group at a board called DRAM at time T0.

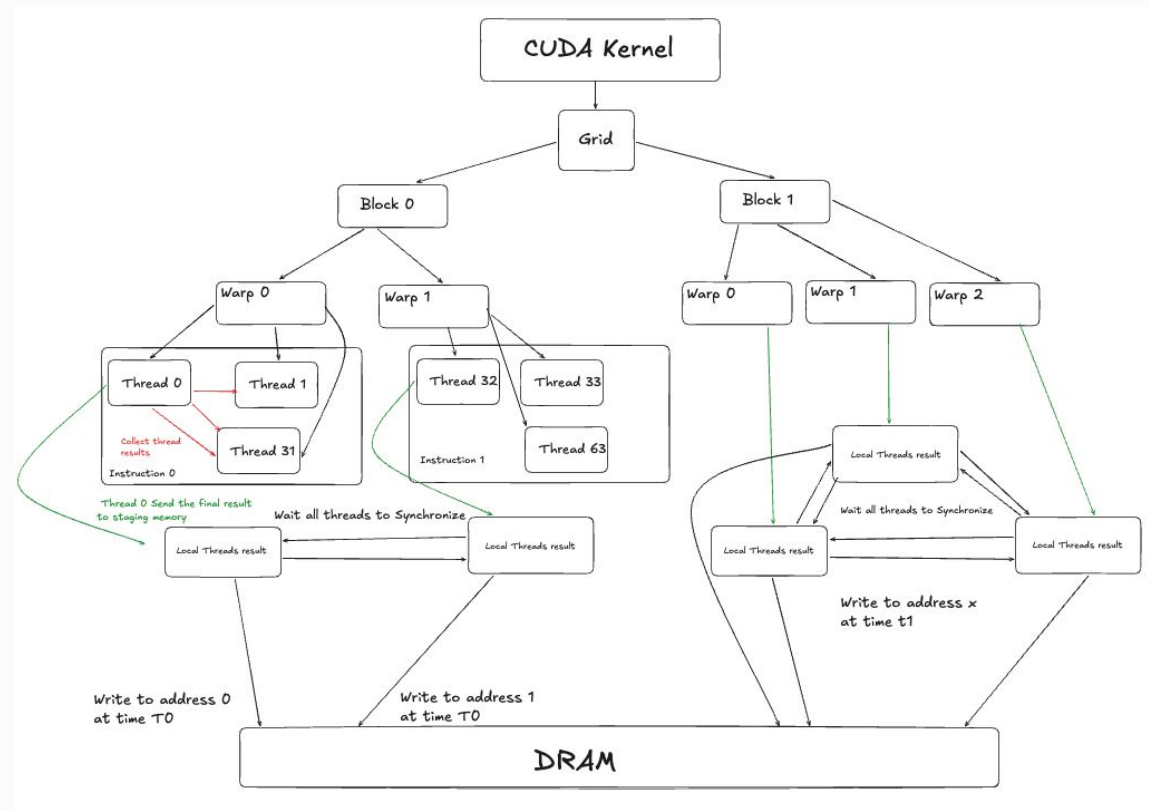
GPU logical execution



But the Block 1 group is not done yet because they have 3 different groups and their performance is lower than Block 0.

A constraint for our problem that Block 0 and Block 1 work is independence so 3 leaders from Block 1 log their result later at time T_1

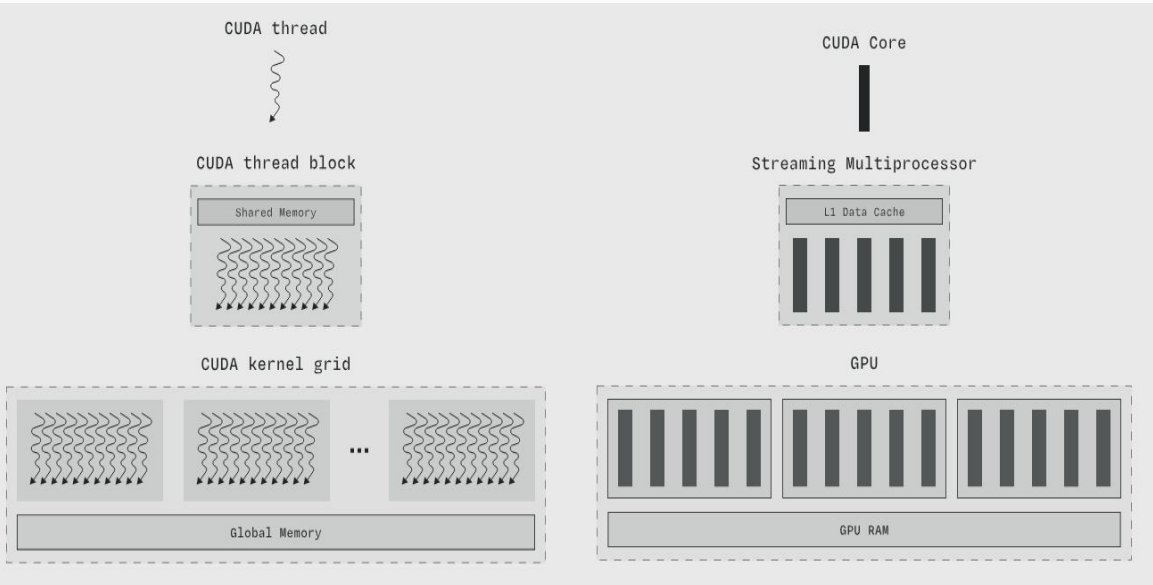
GPU logical execution



To sum it up, this diagram above demonstrate a high level of GPU logical execution where most of task is divide and conquer.

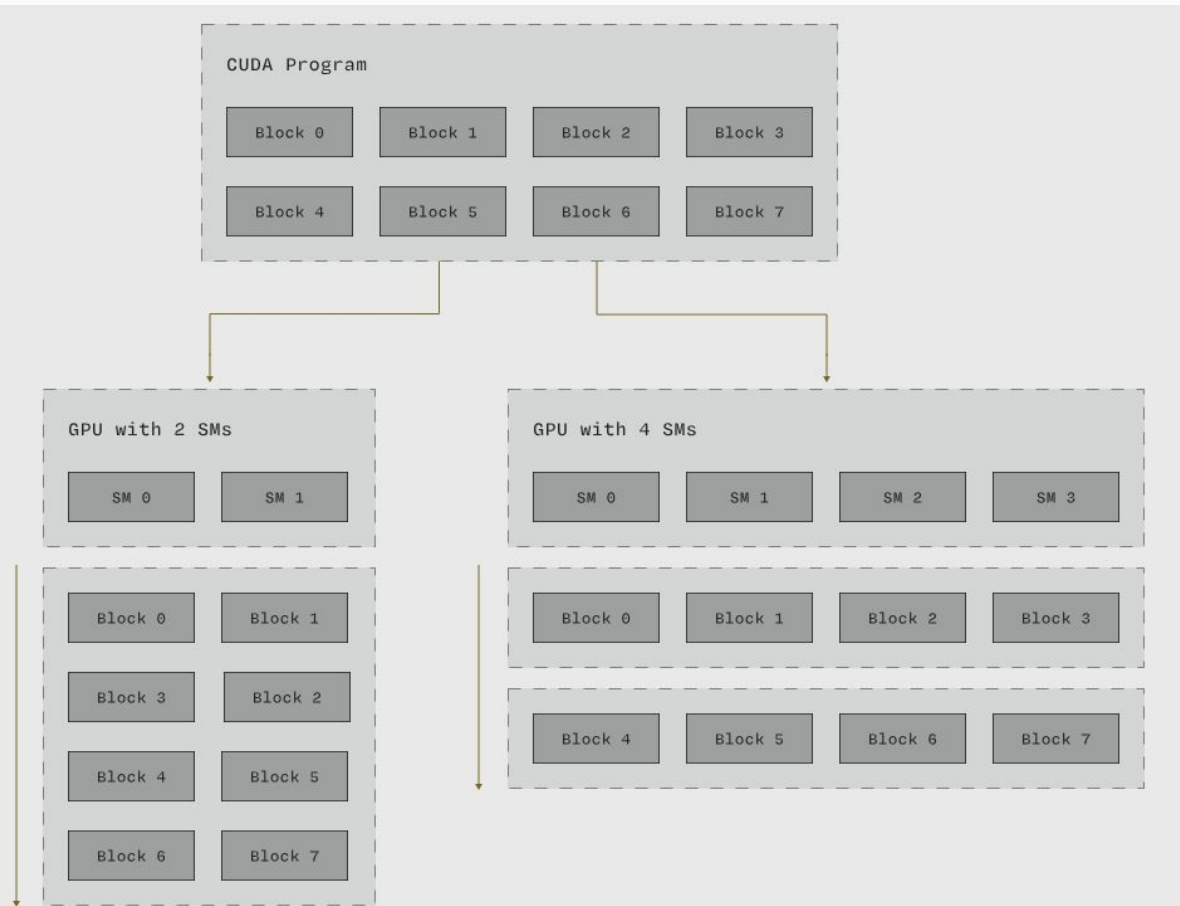
The fundamental logical execution hierarchy in GPU is built upon the idea of Parallelism through divide and conquer and Communication by Collective Operation

Comparison between Logical and Physical Execution model



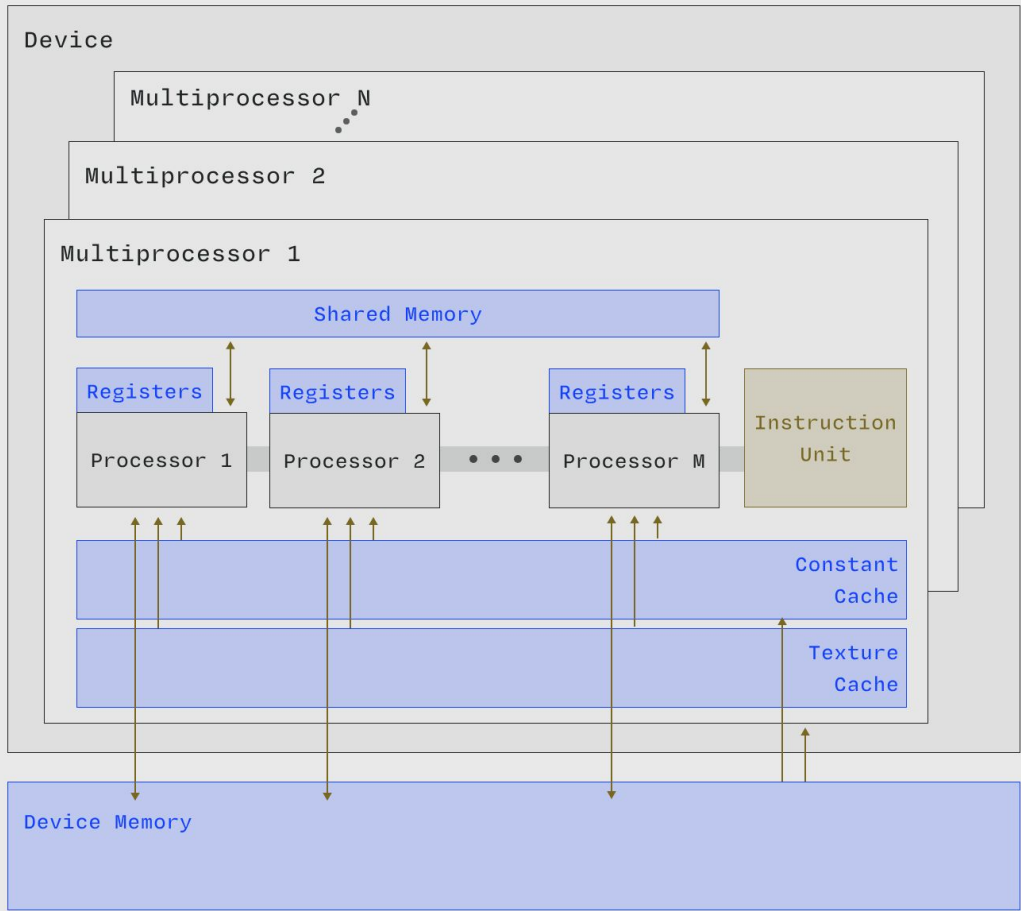
In the physical execution model, each CUDA Core compute a Thread at 1 time while Block is assign to each SM.

Comparison between Logical and Physical Execution model



SM also can only handle 1 block at a time so the idea of parallelism in GPU now add a sequential execution order which block will be compute first.

GPU Logical Execution



As present in this slide, GPU has many SM, each SM now handle computation independently without waiting each other.

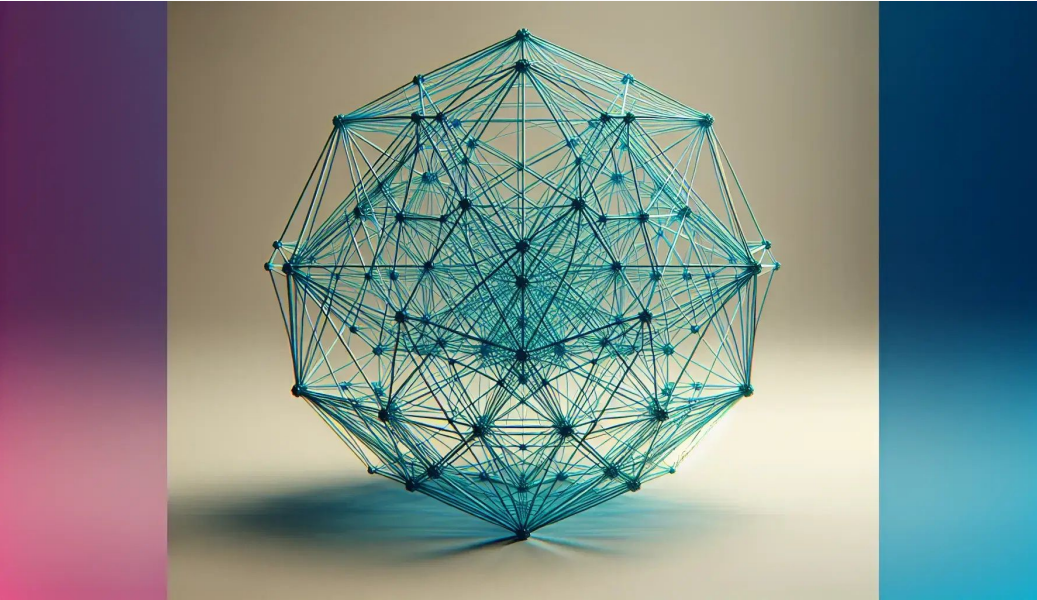
Multi SMs work together in parallel create an army that simultaneously read and write data between DRAM

So how can we manage and orchestrate this workforce to compute ?

The top of the slide features a decorative header with a blue background containing various geometric shapes like triangles and lines in different shades of blue.

GPU Programming Fundamentals

Why CUDA programming exist

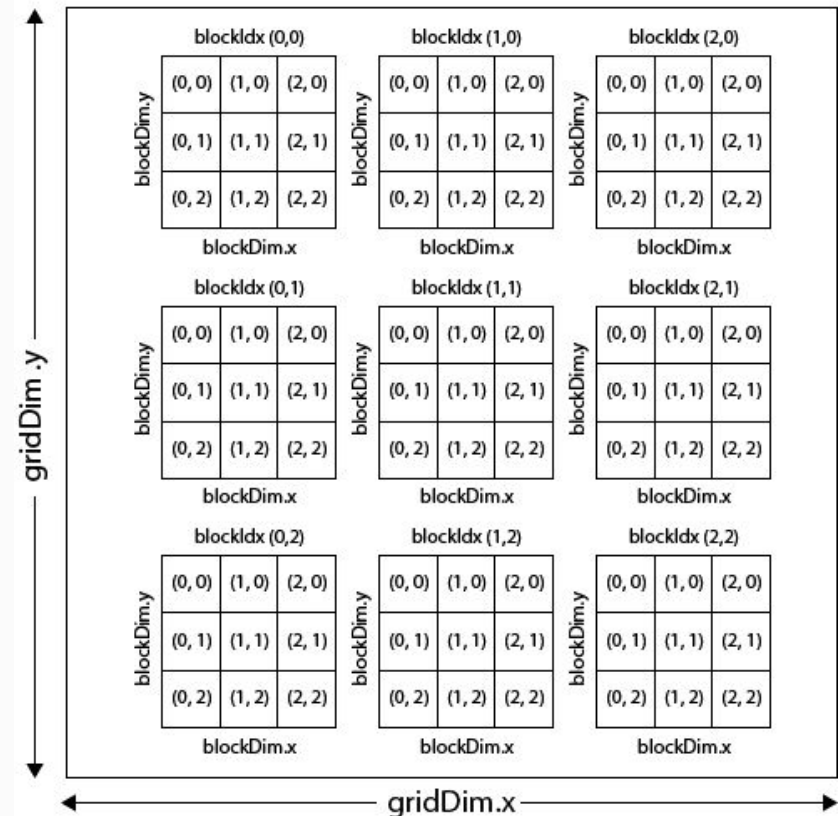


In the computer graphic world, where the GPU is dominant and it is the reason why GPU exist because it job is construct, create and visual the 3D graphic to user

Take a look at this picture, if you have learned linear algebra before you probably can tell each point of this multi-edges shape object have X, Y, and Z coordinate. CUDA design by default using 3d coordinate for Grid, Dim and Thread

CUDA programming

CUDA Grid



Every thread is identified by a logical (x, y, z) coordinate derived from `gridDim`, `blockDim`, `blockIdx`, and `threadIdx`.

This coordinate is a fundamental input in CUDA programming and determines which portion of the data a thread processes.

GPU programing

$$Z = aX + Y$$

To give you a shot about GPU programing, we take a simple calculation example of CUDA core where it only capable of addidction and multiplication

CUDA programing

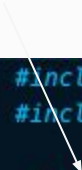
```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

CUDA is implemented on top of C++ syntax, allow host machine to know GPU exist in it run time

This sample code snippet launch a CUDA kernel for Single Precision X Plus Y. It just a sum of X and Y with using same data type (Int, float,etc...)

CUDA programing



```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

The global keyword tell the host machine CUDA is discoverable on the system

CUDA programing

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

The **const** expression before x, and y tell the kernel to READ ONLY X and Y while Z is the output which we want to override in our RAM

CUDA programing

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

We create kernel with thread index position in X dimension, which sit on X dimension of block correspond to the X index dimension of block

CUDA programing

Block 0 (blockIdx.x = 0)

idx 0 threadIdx.x = 0	idx 1 threadIdx.x = 1	idx 2 threadIdx.x = 2	idx 3 threadIdx.x = 3	idx 4 threadIdx.x = 4	idx 5 threadIdx.x = 5	idx 6 threadIdx.x = 6	idx 7 threadIdx.x = 7
-----------------------------	-----------------------------	-----------------------------	-----------------------------	-----------------------------	-----------------------------	-----------------------------	-----------------------------

Block 1 (blockIdx.x = 1)

idx 8 threadIdx.x = 0	idx 9 threadIdx.x = 1	idx 10 threadIdx.x = 2	idx 11 threadIdx.x = 3	idx 12 threadIdx.x = 4	idx 13 threadIdx.x = 5	idx 14 threadIdx.x = 6	idx 15 threadIdx.x = 7
-----------------------------	-----------------------------	------------------------------	------------------------------	------------------------------	------------------------------	------------------------------	------------------------------

- Blocks advance along the X grid dimension
- Threads advance along the X block dimension
- Each thread maps to exactly one data element

CUDA programing

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

This conditional statement is important and because it creates a mask that prevents threads whose global index exceeds the number of elements in the vector from performing work.

Each valid thread multiplies the corresponding element of `x` by the scalar and adds it to the corresponding element of `y`.

The mask allows only threads with valid indices to read and write data. When the number of elements exceeds the number of threads that can execute simultaneously, CUDA schedules different warps onto the cores across next clock cycles.

CUDA launches enough threads to cover the data range, and the conditional mask ensures that threads with invalid indices do not perform any computation or memory access.

CUDA programing

Vector Index	0	1	2	3	4	5	6	7	8	9	10	11
Threads	T0 idx 0	T1 idx 1	T2 idx 2	T3 idx 3	T4 idx 4	T5 idx 5	T6 idx 6	T7 idx 7	T8 idx 8	T9 idx 9	T10 idx 10	T11 idx 11

Legend:

Active thread → performs $y[idx] = a * x[idx] + y[idx]$

Masked thread → condition $idx < n$ fails, no memory access

- CUDA launches enough threads to cover the grid
- Some threads compute $idx \geq n$
- The `if (idx < n)` masks them
- No out-of-bounds read or write occurs
- Remaining elements are handled by **other blocks**, not reused threads

CUDA programing

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(
    int vector_len,
    float a,
    const float *x,
    const float *y,
    float *z
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < vector_len) {
        z[idx] = a * x[idx] + y[idx];
    }
}
```

This conditional statement is important and because it creates a mask that prevents threads whose global index exceeds the number of elements in the vector from performing work.

Each valid thread multiplies the corresponding element of *x* by the scalar and adds it to the corresponding element of *y*.

The mask allows only threads with valid indices to read and write data. When the number of elements exceeds the number of threads that can execute simultaneously, CUDA schedules different warps onto the cores across next clock cycles.

CUDA launches enough threads to cover the data range, and the conditional mask ensures that threads with invalid indices do not perform any computation or memory access.

CUDA Pain



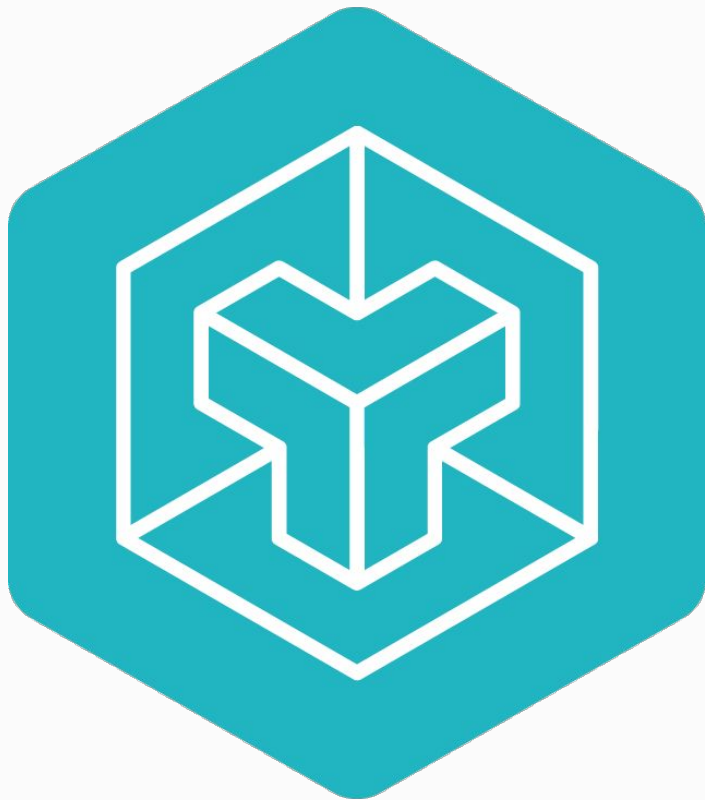
Everyone know C++ is a pain for every programmer to explicit handle memory management, type-checking, etc... This significantly reduce our productivity because we have to spend lots of time to debug error related to C++ principle.

So does CUDA, a language create on top off C++ make it less friendly for everyone, so OpenAI create Triton to program GPU purely in Python.

Triton

Triton is a high level DSL and hardware agnostic for GPU programming introduced by OpenAI. Triton follow these design principles

- Give AI Researcher time to modify kernel as it need without memory management, type-checking in CUDA (based on C++)
- Device agnostic for different hardware such as AMD Rocm, NVIDIA CUDA and XLA based like Google TPU, AWS Trainium.
- Only manage how each block is scheduled, tiled inside a grid, completely ignore thread.



Triton

```
1 import triton
2 import triton.language as tl
3
4 @triton.jit
5 def saxpy(
6     x_ptr,          # pointer to x
7     y_ptr,
8     # pointer to y (in-place output). Different from CUDA,
9     #this must be inplace output where CUDA we can either inplace or assign to new objec
10    a,               # scalar
11    n,               # number of elements
12    BLOCK_SIZE: tl.constexpr,
13 ):
14     # Program id along X dimension, Block is Triton lowest compute level, instead of Thread in CUDA
15     pid = tl.program_id(0)
16
17     # Offsets inside this program. This operation define number of memory address for your block kernel.
18     offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
19
20     # Mask to prevent out-of-bounds access. This equal to the if condition we mention in CUDA for memory safety
21     mask = offsets < n
22
23     # Load
24     x = tl.load(x_ptr + offsets, mask=mask)
25     y = tl.load(y_ptr + offsets, mask=mask)
26
27     # Compute
28     y = a * x + y
29
30     # Store
31     tl.store(y_ptr + offsets, y, mask=mask)
```

This is the saxpy kernel implemented in Triton, no C++ boilerplate more.

But to remove the language boundary, there is a price to pay which now the lowest level of GPU Logical execution we can control is Block

Triton

```
1 import triton
2 import triton.language as tl
3
4 @triton.jit
5 def saxpy(
6     x_ptr,          # pointer to x
7     y_ptr,          # pointer to y (in-place output). Different from CUDA,
8                     #this must be inplace output where CUDA we can either inplace or assign to new objec
9     a,              # scalar
10    n,               # number of elements
11    BLOCK_SIZE: tl.constexpr,
12 ):
13
```

First, we wrap the saxpy kernel in triton JIT decorator, this will compile our python code in to lower level representation (If you want to know the compilation process, read [this](#))

The argument is still same but now there are a new arg called BLOCK_SIZE which define the number of thread we want our block to handle

Triton

```
# Program id along X dimension, Block is Triton lowest compute level, instead of Thread in CUDA
pid = tl.program_id(0)

# Offsets inside this program. This operation define number of memory address for your block kernel.
offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
```

The offsets is equivalent to this operation in CUDA where you specify number of thread in your block

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

The pid object instantiate a block, in Triton called program

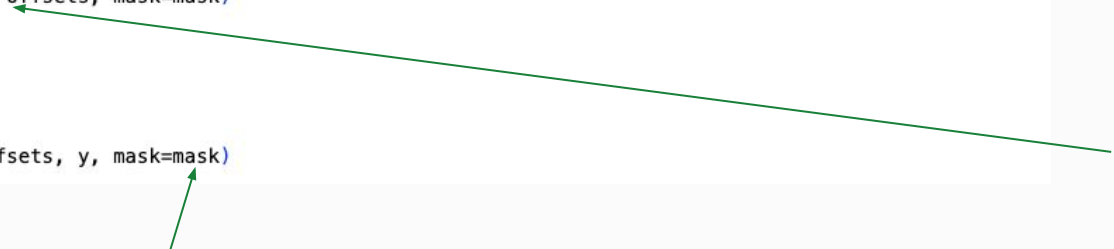
Triton

```
# Mask to prevent out-of-bounds access. This equal to the if condition we mention in CUDA for memory safety
mask = offsets < n

# Load
x = tl.load(x_ptr + offsets, mask=mask)
y = tl.load(y_ptr + offsets, mask=mask)

# Compute
y = a * x + y

# Store
tl.store(y_ptr + offsets, y, mask=mask)
```



The overflow Tensor will send to the new block for calculation, if the number of Tensor in this block is less than the number of offsets in new block, we mask them for memory safety

Triton does not remove the hard part which is the memory masking which we have to define like CUDA.

After create mask, we have to load the Tensor with corresponding pointer and the offsets. You can think that the pointer is how Triton access the memory of the Tensor and the offsets above is to handle overflow Tensor in Triton.

Program ID

```
1 @triton.jit
2 def add_3d_simple_kernel(
3     x_ptr, y_ptr, out_ptr,
4     D, H, W,
5     BLOCK_D: tl.constexpr,
6     BLOCK_H: tl.constexpr,
7     BLOCK_W: tl.constexpr
8 ):
9     pid_d = tl.program_id(axis = 0) # Manage width dimension of a block
10    pid_h = tl.program_id(axis = 1) # Manage length dimension of a block
11    pid_w = tl.program_id(axis = 2) # Manage height dimension of a block
```

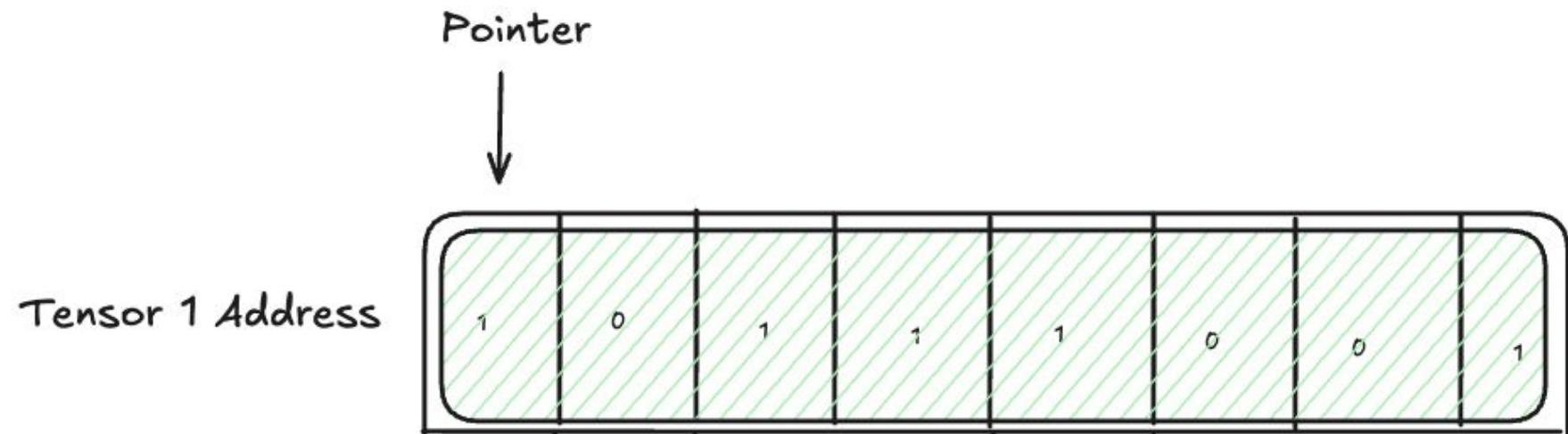
Triton still retain the 3D programming principle from CUDA where you can manage all dimensions of your block

Mapping, Masking

- Pointer: Point where the memory address
- Mask: When launch a kernel in Triton, we define a max number of memory address in a Block
- Offset: Tensor memory indices in Block

If number of Tensor > Offset => Overflow address is masked in this Block

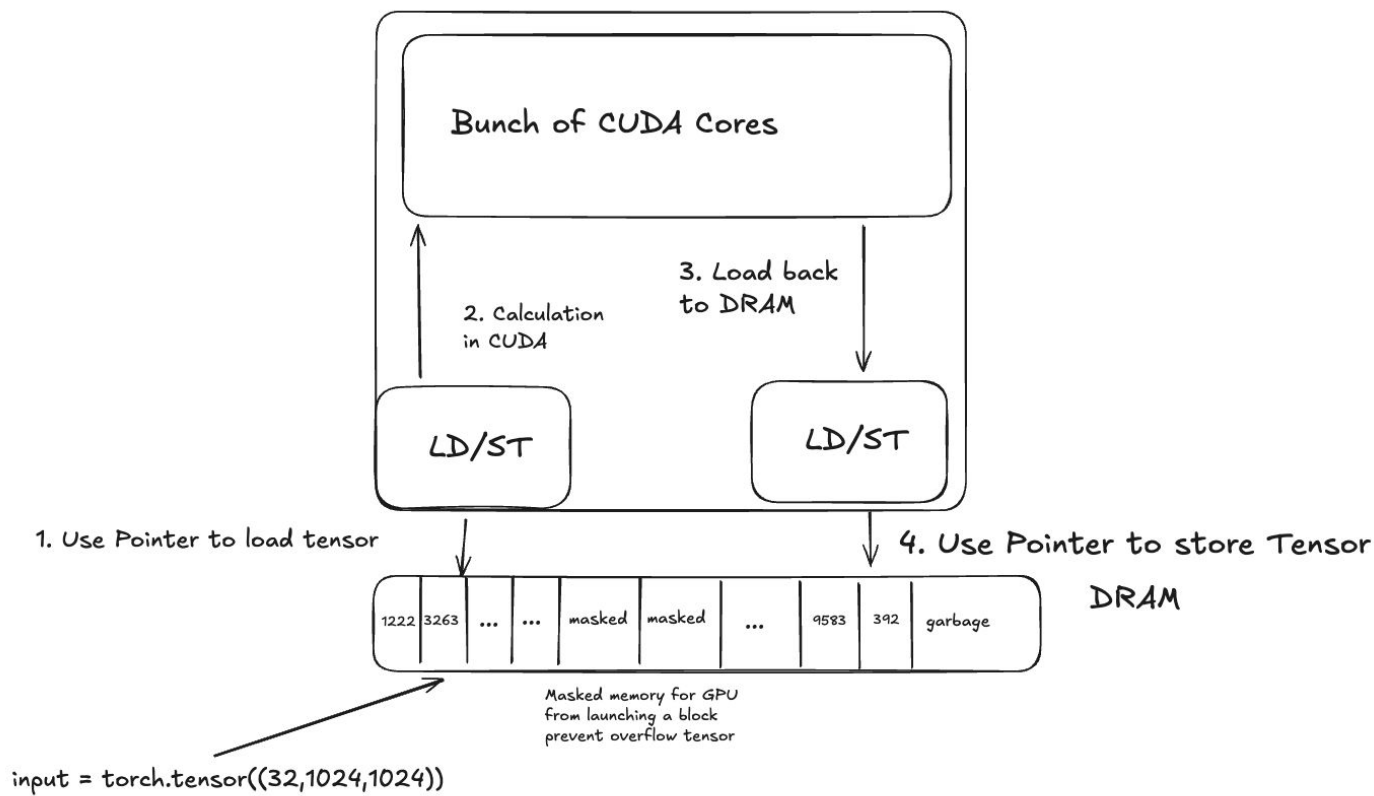
Mapping, Masking



Offset which is covered in green depict the address of the Tensor 1 in DRAM

Mapping, Masking

High level flow of how Tensor is load from DRAM to CUDA using pointer



Mapping, Masking

```
1 @triton.jit
2 def add_kernel(x_ptr, # *Pointer* to first input vector.
3               y_ptr, # *Pointer* to second input vector.
4               output_ptr, # *Pointer* to output vector.
5               n_elements, # Size of the vector.
6               BLOCK_SIZE: tl.constexpr, # Number of elements each program should process.
7               # NOTE: `constexpr` so it can be used as a shape value.
8               ):
9     # There are multiple 'programs' processing different data. We identify which program
10    # we are here:
11    pid = tl.program_id(axis=0) # We use a 1D launch grid so axis is 0.
12    # This program will process inputs that are offset from the initial data.
13    # For instance, if you had a vector of length 256 and block_size of 64, the programs
14    # would each access the elements [0:64, 64:128, 128:192, 192:256].
15    # Note that offsets is a list of pointers:
16    block_start = pid * BLOCK_SIZE
17    offsets = block_start + tl.arange(0, BLOCK_SIZE)
18    # Create a mask to guard memory operations against out-of-bounds accesses.
19    mask = offsets < n_elements
20    # Load x and y from DRAM, masking out any extra elements in case the input is not a
21    # multiple of the block size.
22    x = tl.load(x_ptr + offsets, mask=mask)
23    y = tl.load(y_ptr + offsets, mask=mask)
24    output = x + y
25    # Write x + y back to DRAM.
26    tl.store(output_ptr + offsets, output, mask=mask)
```

Mapping, Masking

```
1 def add(x: torch.Tensor, y: torch.Tensor):
2     # We need to preallocate the output.
3     output = torch.empty_like(x)
4     n_elements = output.numel()
5
6     # triton need meta-params for GPU hardware such as BLOCK_SIZE, num_process, etc... the jit function accept meta-params with tl.constexpr
7     grid = lambda meta: (triton.cdiv(n_elements, meta['BLOCK_SIZE']), ) # we launch grid shape equal to ceil of num output elements and block size
8     # for example numel = 3242 and BLOCK_SIZE = 1024 -> we launch grid with shape (324,1024,1) equal to the gridDim at CUDA
9     # NOTE:
10    # - Each torch.tensor object is implicitly converted into a pointer to its first element.
11    # - `triton.jit`'ed functions can be indexed with a launch grid to obtain a callable GPU kernel.
12    # - Don't forget to pass meta-parameters as keywords arguments.
13    add_kernel[grid](x, y, output, n_elements, BLOCK_SIZE=1024)
14    # We return a handle to z but, since `torch.cuda.synchronize()` hasn't been called, the kernel is still
15    # running asynchronously at this point.
16    return output
```

The JIT function compile our previous kernel function, so we pass the meta-parameters and input to the kernel function

Can you demonstrate the execution hierarchy again for a Vector computation at this slide.

Striding

Suppose that our data is now 2D
Tensor, this is how its memory
address is

Tensor 1 Row Address	1	0	1	1	1	0	0	1
Tensor 1 Column Address	1	0	0	0	0	0	0	1

For visualization, the memory is shaped to 2D, in reality, memory shape is 1D

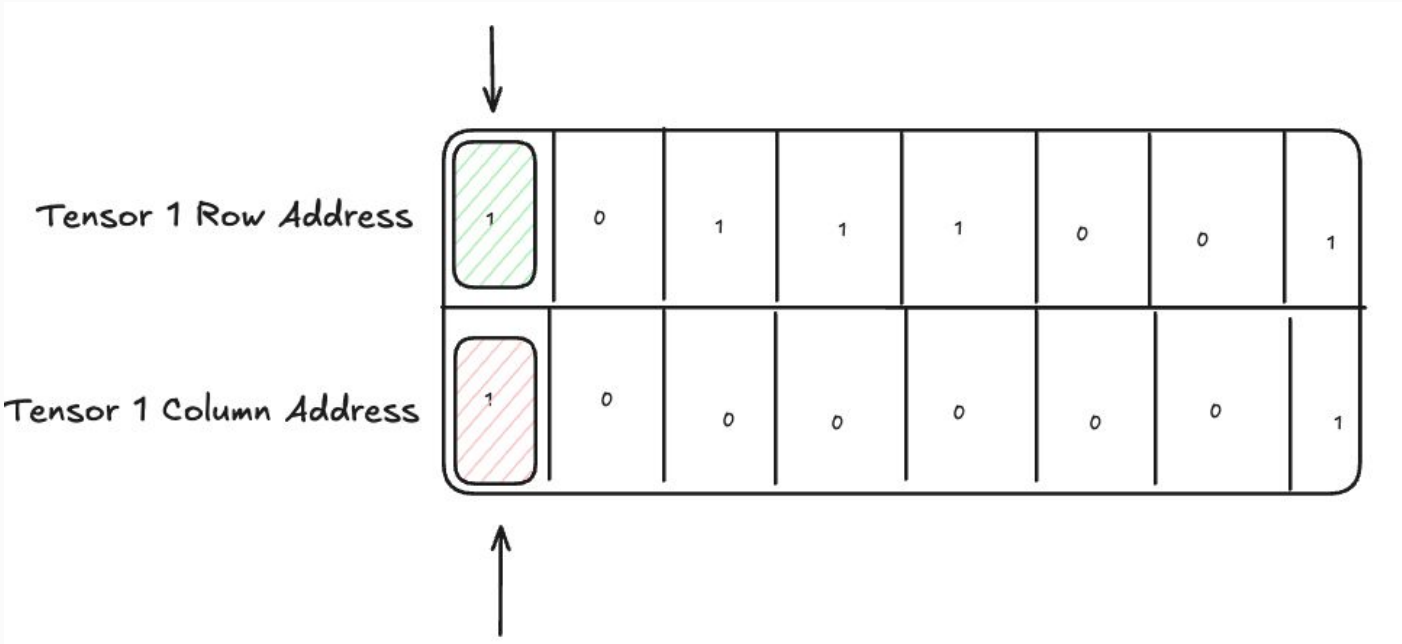
Striding

Tensor 1 Row Address	1	0	1	1	1	0	0	1
Tensor 1 Column Address	1	0	0	0	0	0	0	1

We want to load this Tensor columnar-wise for example first element of row and column at one

For visualization, the memory is shaped to 2D, in reality, memory shape is 1D

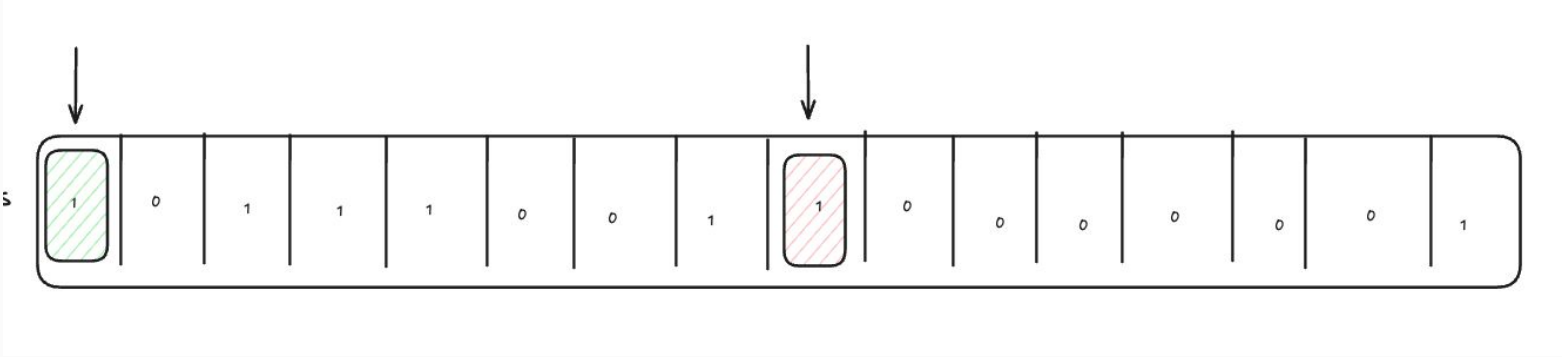
Striding



So we need 2 distinct pointers, each point at the first element of row and column of the Tensor

Striding

But in fact, the memory address is a 1D array, so how can we keep track of our pointer ?



Striding

```
1 import triton
2 import triton.language as tl
3
4 @triton.jit
5 def block_load_kernel(
6     pointer,          # The starting address of the matrix in memory
7     stride_row,       # How many elements to skip to get to the next row
8     BLOCK_SIZE_M: tl.constexpr,
9     BLOCK_SIZE_N: tl.constexpr,
10 ):
11     pid = tl.program_id(0)
12
13     # 1. Create a range of indices for rows and columns
14     # Example: [0, 1, 2, 3... 15]
15     offs_m = tl.arange(0, BLOCK_SIZE_M)
16     offs_n = tl.arange(0, BLOCK_SIZE_N)
17
18     # 2. Use STRIDES to turn 2D indices into 1D memory addresses
19     # We multiply the row indices by the row stride.
20     # We add the column indices (assuming column stride is 1).
21     # [BLOCK_SIZE_M, 1] * [1, BLOCK_SIZE_N] creates a 2D grid of pointers
22     element_pointers = pointer + (offs_m[:, None] * stride_row + offs_n[None, :] * 1)
23
24     # 3. Load the block of data
25     block_data = tl.load(element_pointers)
26
```

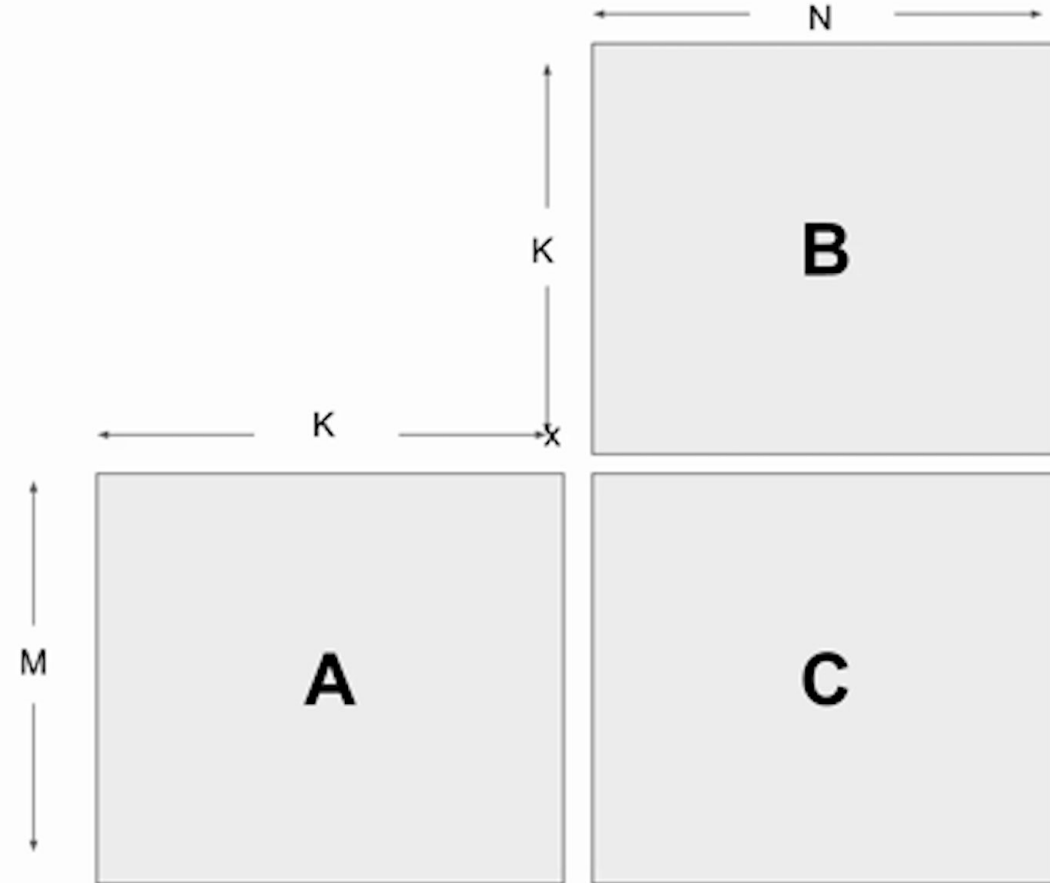
We create 2 offsets for row and column and the element pointer to simply load the tensor just like we want

Striding

```
# 2. Use STRIDES to turn 2D indices into 1D memory addresses
# We multiply the row indices by the row stride.
# We add the column indices (assuming column stride is 1).
# [BLOCK_SIZE_M, 1] * [1, BLOCK_SIZE_N] creates a 2D grid of pointers
element_pointers = pointer + (offs_m[:, None] * stride_row + offs_n[None, :] * 1)
```

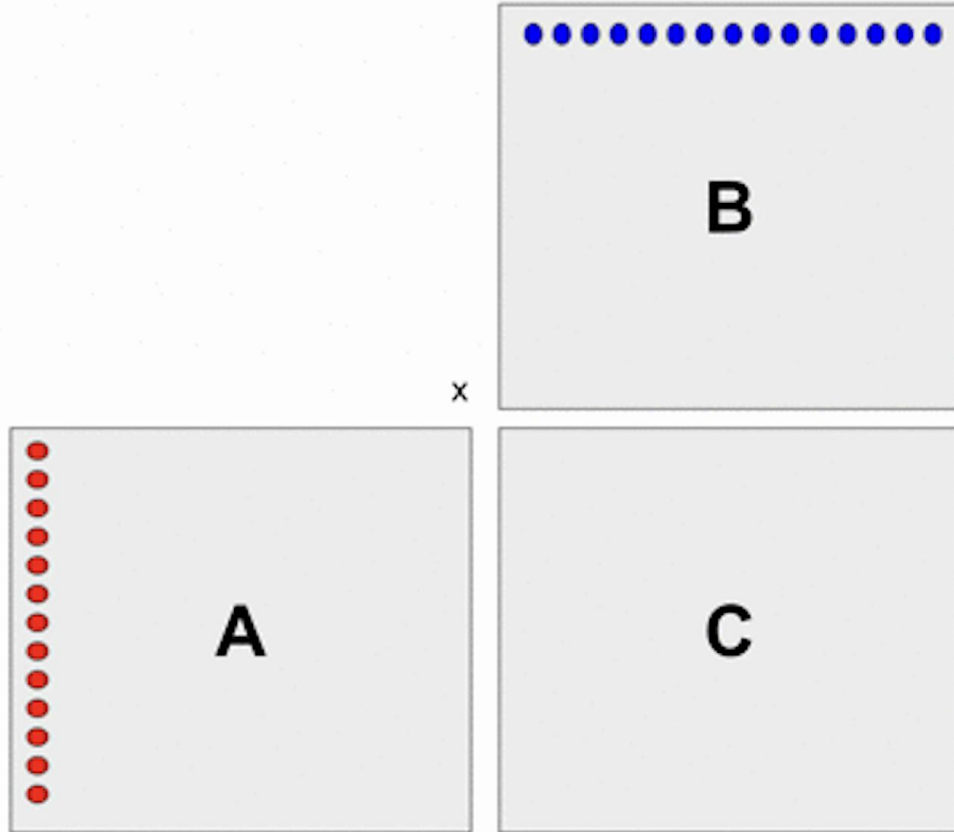
This line create all pointers correspond to the elements of the matrix which we want, it look like this

Matrix Multiplication



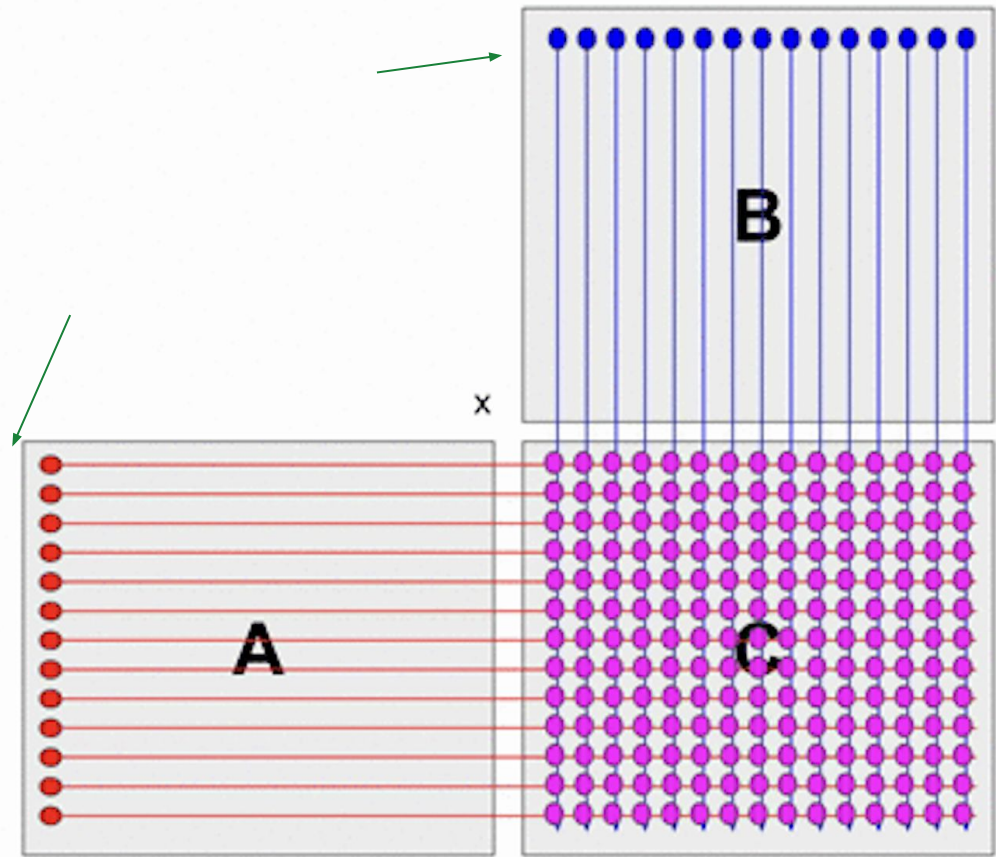
We have 2 matrix, $A(M \times K)$ and $B(K \times N)$. So the C matrix have shape $(M \times N)$.

Matrix Multiplication



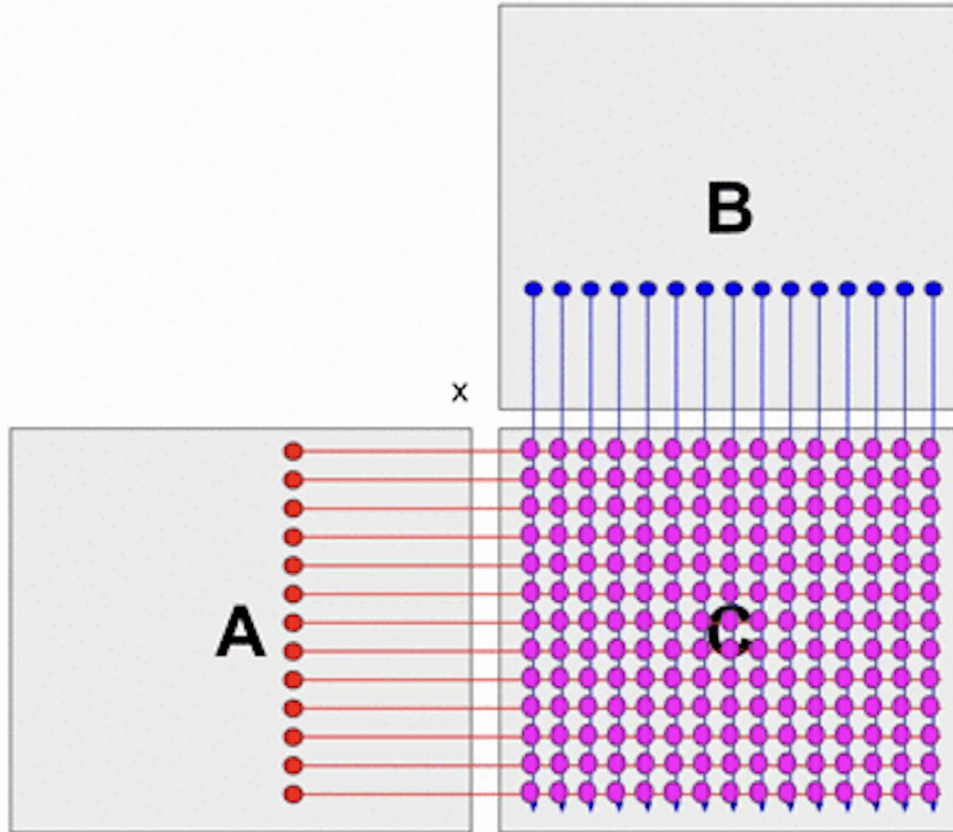
We begin to select each element in the 1st dimension of matrix A and the 2nd dimension of matrix B

Matrix Multiplication



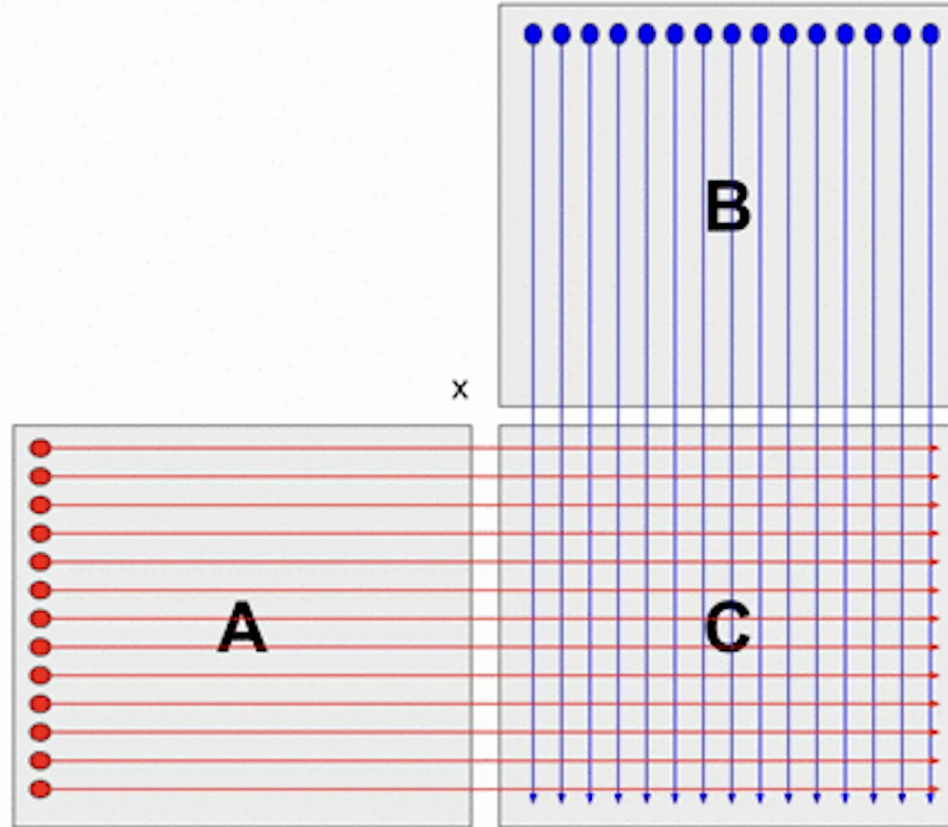
Iterate all the row and column, this simple matmul calculation depict we calculate dot product between row and column with correspond index of each matrix in GPU programing

Matrix Multiplication



Iterate all the row and column,
this simple matmul calculation
depict we calculate dot product
between row and column with
correspond index of each
matrix in GPU programing

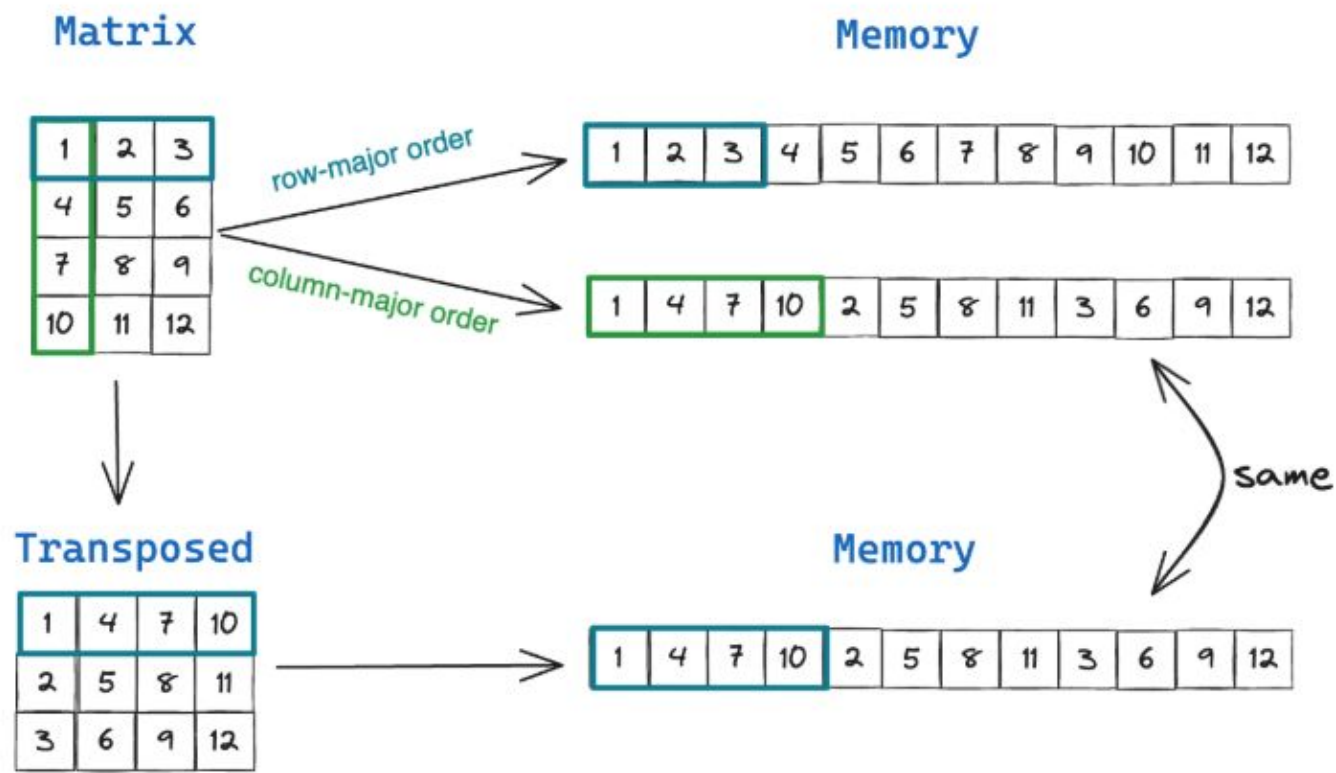
Effective Matrix Multiplication



But computer itself know only to read the matrix in either row-wise or column-wise, so how can we perform this iteration as we said above ?

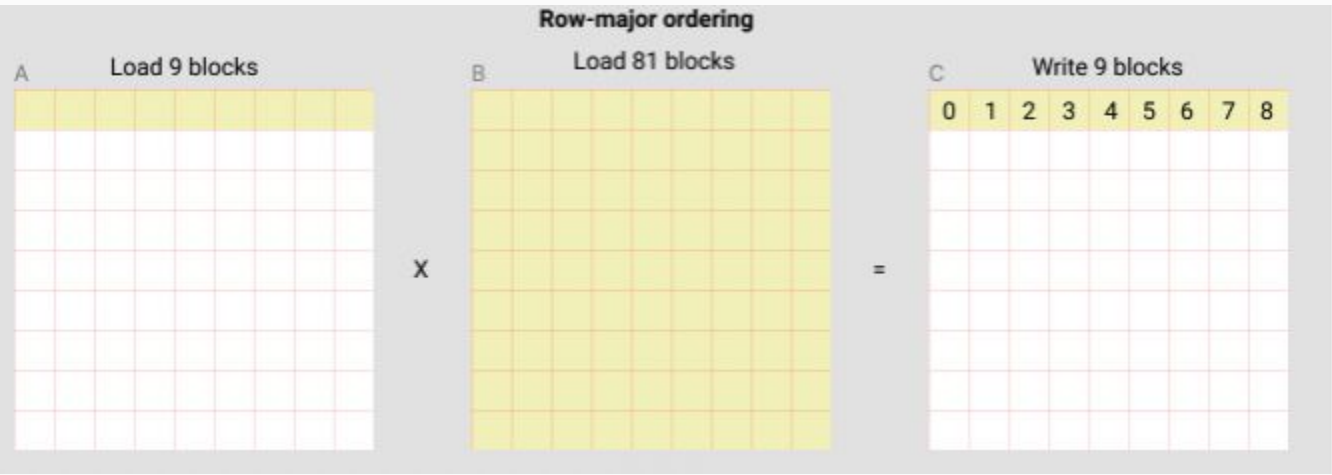
Also, this operation is ineffective because we have to load entire matrix A or matrix B because of the row or column read order.

Effective Matrix Multiplication



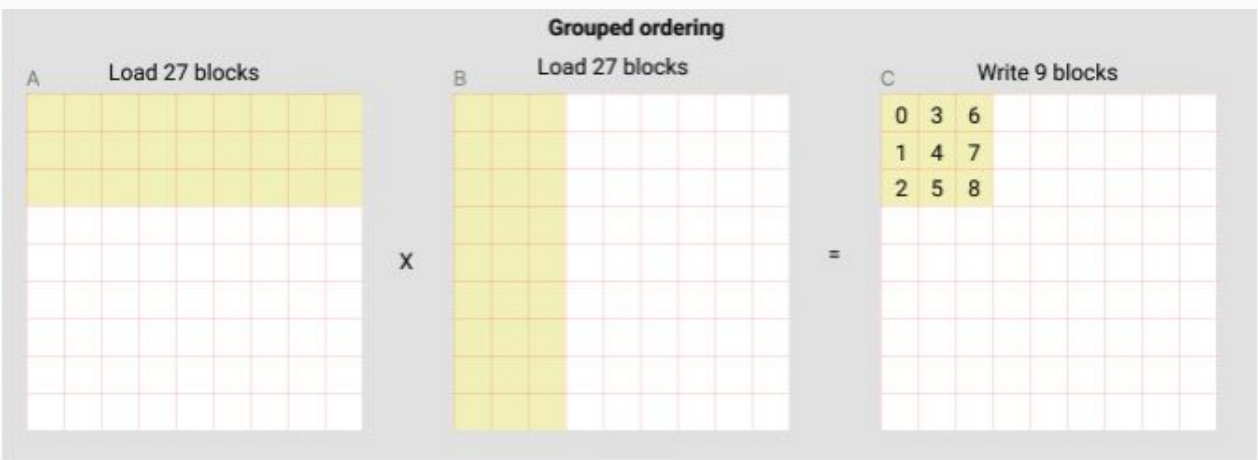
In C++ or default Python, we use Row-major order matrix. But for analytics we prefer column to be Column-major order for analytics application

Effective Matrix Multiplication



In traditional Row-major ordering, to compute Matmul, we have to read 9 block from A and whole B block to calculate the 1st row of output matrix. It's ineffective because we have to load entire matrix B just for a line of calculation in matrix C.

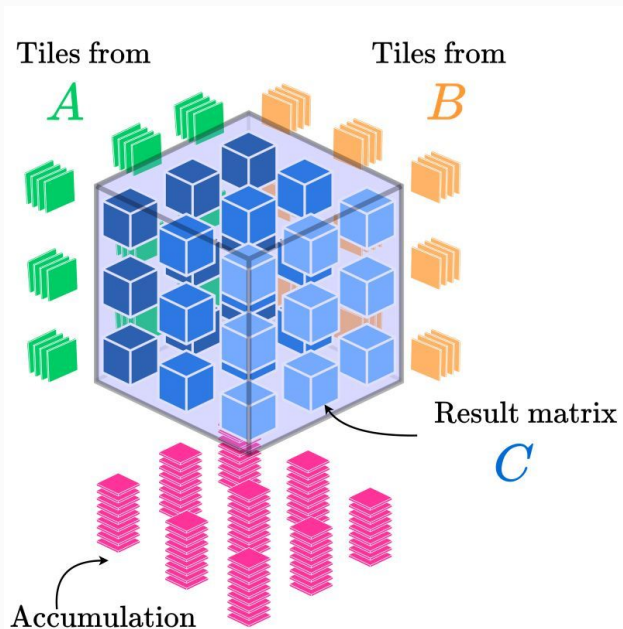
Effective Matrix Multiplication



This is where big brain meet, we need to use both of row and column major order to form a Group ordering where only a portion of matrix C is calculate during one iteration. This is called **GEMM**(General Matrix Multiplication) operations in GPU Programing.

This strategy is effective because we load the exact number of element in both matrix A and matrix B

Effective Matrix Multiplication



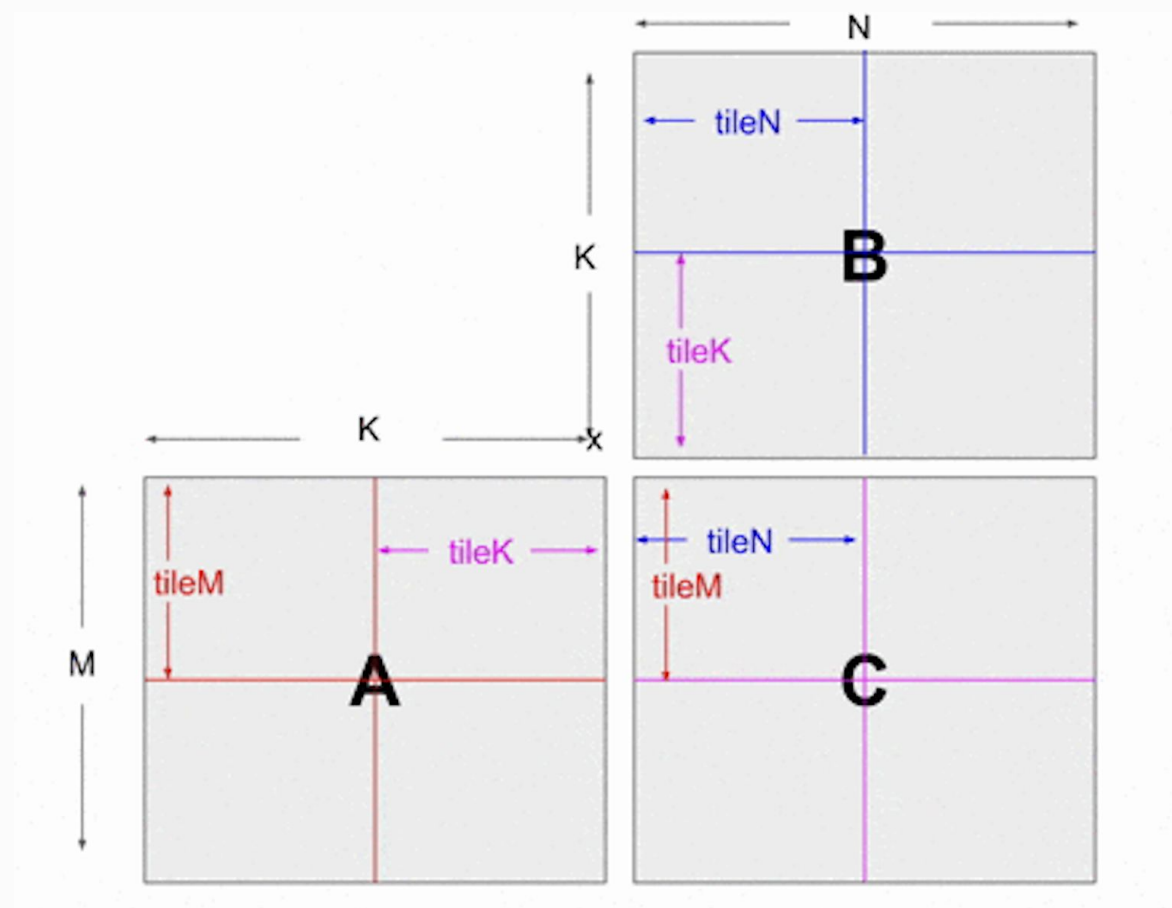
$$C_{i,j} = \sum_k A_{i,k} \times B_{k,j}$$

To implement the group matmul, in GPU programming, this is called Tiling

Tiling GPU can be know as divide thread blocks, grids, and wraps from logical execution to SM in physical execution

The accumulation is an empty Tensor where it predefine address for matmul.

Effective Matrix Multiplication

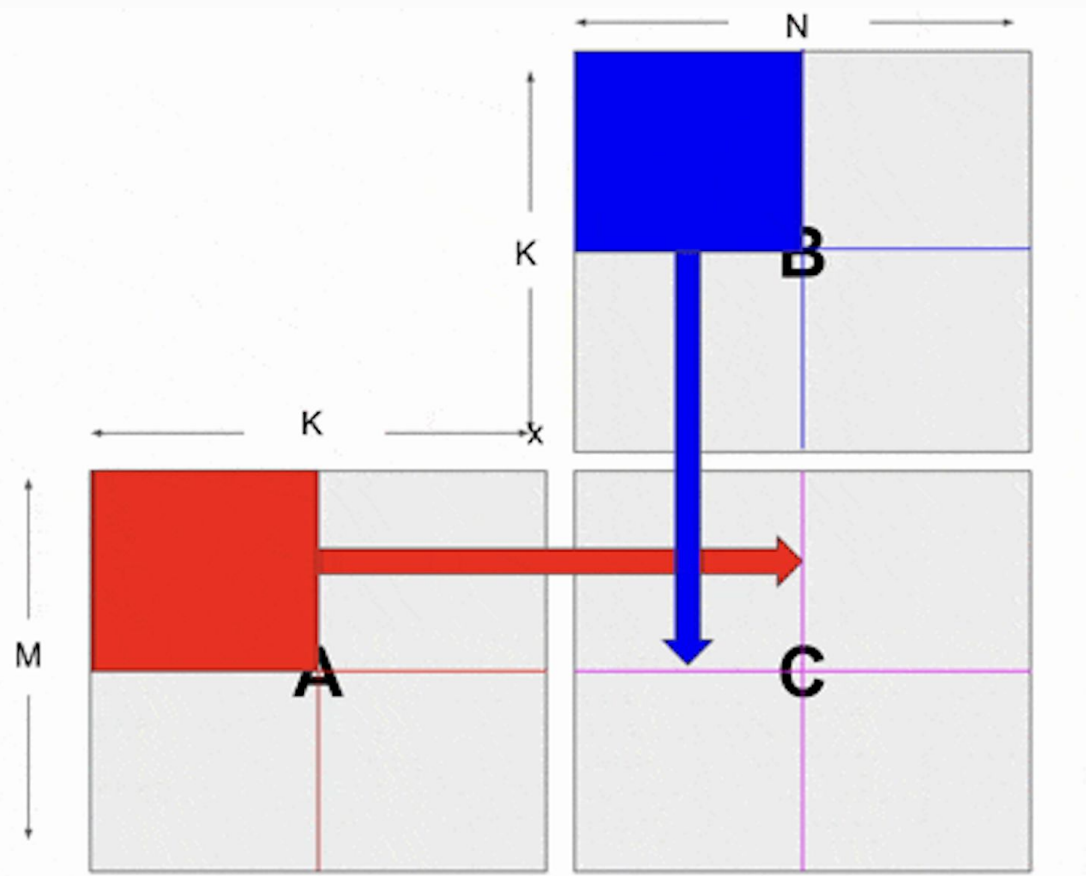


To deal with this, GEMM is introduced to handle matmul using tile

Each tile is a subset of of correspond dimension from A and B matrix:

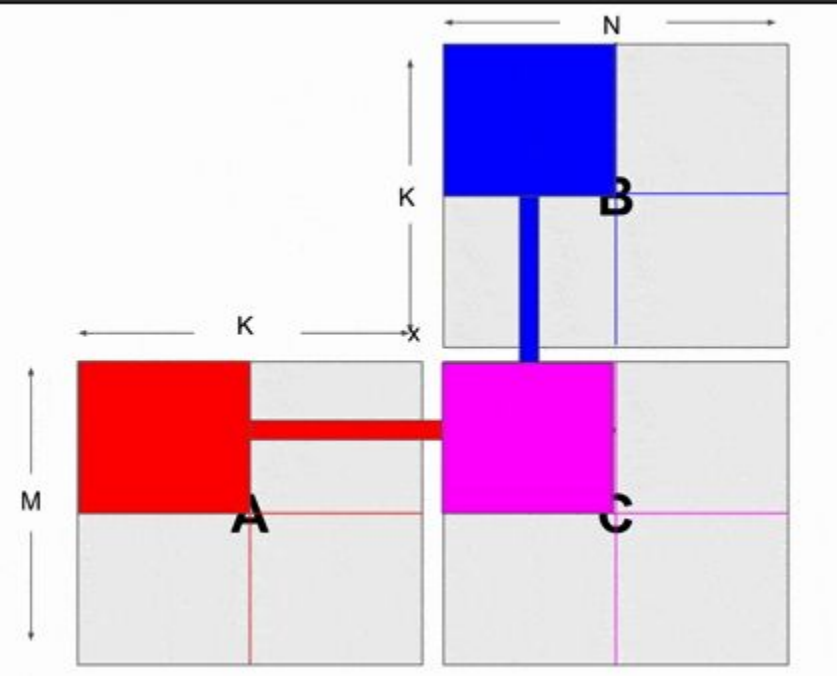
- Tile M: Subset of M dim in matrix A
- Tile N: Subset of N dim in matrix B
- Tile K: Output tile of result matrix

Effective Matrix Multiplication



We have total 4 tiles so each block of the matrix C is compute by the corresponding tile from A and B

Effective Matrix Multiplication



The iteration now drop into only 4 iterations only. So the number of iteration depend on tiles while the shape of tile depend on how big your kernel is so it varies by GPU hardware.

Effective Matrix Multiplication

```
1 @triton.jit
2 def matmul_kernel(
3     # Pointers to matrices
4     a_ptr, b_ptr, c_ptr,
5     # Matrix dimensions
6     M, N, K,
7     # Strides
8     stride_am, stride_ak,
9     stride_bk, stride_bn,
10    stride_cm, stride_cn,
11    # Meta-parameters
12    BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr, BLOCK_SIZE_K: tl.constexpr,
13):
14    # 1. Launch 2d block
15    pid_m = tl.program_id(0)
16    pid_n = tl.program_id(1)
17
18    # 2. Create offset for each vector
19    offs_m = (pid_m * BLOCK_SIZE_M) + tl.arange(0, BLOCK_SIZE_M) # this will be matrix offset for M at dim 0
20    offs_n = (pid_n * BLOCK_SIZE_N) + tl.arange(0, BLOCK_SIZE_N) # this will be matrix offset for N at dim 1
21    offs_k = tl.arange(0, BLOCK_SIZE_K) # we dont use pid for the output
22
23    acc = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
24
25    # 4. Iterate through K to calculate dot product for each tile
26    for k in range(0, K, BLOCK_SIZE_K): # over K dimension with step is BLOCK_SIZE_K to jump to next tile
27        # Update current K indices for this iteration
28        curr_k = k + offs_k
29
30        # Create 2D pointer blocks by broadcasting M and K indices
31        # as a tile, the a pointers will point to the 1st dim of M offset
32        # and 2nd dim of K offset because the M denote the address of row and K denote the address of column in A matrix
33
34        # The multiply by stride for correct memory offset
35        # This implementation only the k move dynamically where the k is incremented in each loop
36        # Shape of a_ptrs: (BLOCK_SIZE_M, BLOCK_SIZE_K)
37        a_ptrs = a_ptr + (offs_m[:, None] * stride_am + curr_k[None, :] * stride_ak)
38
39        # Shape of b_ptrs: (BLOCK_SIZE_K, BLOCK_SIZE_N)
40        b_ptrs = b_ptr + (curr_k[:, None] * stride_bk + offs_n[None, :] * stride_bn)
41
42        # Load blocks with masking to handle cases where M, N, or K aren't multiples of block sizes
43        a_block = tl.load(a_ptrs, mask=(offs_m[:, None] < M) & (curr_k[None, :] < K))
44        b_block = tl.load(b_ptrs, mask=(curr_k[:, None] < K) & (offs_n[None, :] < N))
45
46        # Matrix multiplication on the blocks
47        acc += tl.dot(a_block, b_block)
48
49    # 6. Store the final result back to Global Memory
50    # The pointer grid must match the shape of 'acc' (BLOCK_SIZE_M, BLOCK_SIZE_N)
51    c_ptrs = c_ptr + (offs_m[:, None] * stride_cm + offs_n[None, :] * stride_cn)
52    c_mask = (offs_m[:, None] < M) & (offs_n[None, :] < N)
53
54    tl.store(c_ptrs, acc, mask=c_mask)
```

We instantiate 2D block for our kernel which each block dim represent to row and col of input matrix.

The naive implementation for tiling which is iterate over K, using BLOCK_SIZE_K as a stride to mimic the mask in the DRAM.

The pointer for a and b will be shifted during calculation cause the k pointer is incremented by 1

Append the accumulate result to the accumulator

Effective Matrix Multiplication

```
1 @triton.jit
2 def matmul_kernel(
3     # Pointers to matrices
4     a_ptr, b_ptr, c_ptr,
5     # Matrix dimensions
6     M, N, K,
7     stride_am, stride_ak, #
8     stride_bk, stride_bn, #
9     stride_cm, stride_cn,
10    # Meta-parameters
11    BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr,
12    GROUP_SIZE_M: tl.constexpr #
13 ):
14     """Kernel for computing the matmul C = A x B.
15     A has shape (M, K), B has shape (K, N) and C has shape (M, N)
16     """
17     # -----
18     # Map program ids 'pid' to the block of C it should compute.
19     # This is done in a grouped ordering to promote L2 data reuse.
20     # See above 'L2 Cache Optimizations' section for details.
21     pid = tl.program_id(axis=0)
22     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
23     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
24     num_pid_in_group = GROUP_SIZE_M * num_pid_n
25     group_id = pid // num_pid_in_group
26     first_pid_m = group_id * GROUP_SIZE_M
27     group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
28     pid_m = first_pid_m + (pid % num_pid_in_group) % group_size_m
29     pid_n = (pid % num_pid_in_group) // group_size_m
30
31     # -----
32     # Create pointers for the first blocks of A and B.
33     # We will advance this pointer as we move in the K direction
34     # and accumulate
35     # 'a_ptrs' is a block of [BLOCK_SIZE_M, BLOCK_SIZE_K] pointers
36     # 'b_ptrs' is a block of [BLOCK_SIZE_K, BLOCK_SIZE_N] pointers
37     # See above 'Pointer Arithmetic' section for details
38     offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
39     offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
40     offs_k = tl.arange(0, BLOCK_SIZE_K)
41     a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak)
42     b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn)
43
44     # -----
45     # Iterate to compute a block of the C matrix.
46     # We accumulate into a '[BLOCK_SIZE_M, BLOCK_SIZE_N]' block
47     # of fp32 values for higher accuracy.
48     # 'accumulator' will be converted back to fp16 after the loop.
49     accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N), dtype=tl.float32)
50     for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
51         # Load the next block of A and B, generate a mask by checking the K dimension.
52         # If it is out of bounds, set it to 0.
53         a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k * BLOCK_SIZE_K, other=0.0)
54         b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k * BLOCK_SIZE_K, other=0.0)
55         # We accumulate along the K dimension.
56         accumulator = tl.dot(a, b, accumulator)
57         # Advance the ptrs to the next K block.
58         a_ptrs += BLOCK_SIZE_K * stride_ak
59         b_ptrs += BLOCK_SIZE_K * stride_bk
60
61     # -----
62     # Write back the block of the output matrix C with masks.
63     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
64     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
65     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn * offs_cn[None, :]
66     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
67     tl.store(c_ptrs, c, mask=c_mask)
```

From the official Triton documentation, we see that the implementation is different by flatten the M and N to a single block dimension

The idea of striding to allocate dynamic pointer is same as our implementation.

Effective Matrix Multiplication

```
1 def matmul(a, b, kernel):
2     M, K = a.shape
3     K, N = b.shape
4     c = torch.empty((M, N), device=a.device, dtype=a.dtype)
5     grid = lambda META: (
6         triton.cdiv(M, META['BLOCK_SIZE_M']),
7         triton.cdiv(N, META['BLOCK_SIZE_N'])
8     )
9     if kernel == "our":
10         matmul_kernel[grid](
11             a, b, c,
12             M, N, K,
13             a.stride(0), a.stride(1),
14             b.stride(0), b.stride(1),
15             c.stride(0), c.stride(1),
16             BLOCK_SIZE_M=64, BLOCK_SIZE_N=64, BLOCK_SIZE_K=32,
17         )
18     else:
19         matmul_kernel_openai[grid](
20             a, b, c,
21             M, N, K,
22             a.stride(0), a.stride(1),
23             b.stride(0), b.stride(1),
24             c.stride(0), c.stride(1),
25             BLOCK_SIZE_M=64, BLOCK_SIZE_N=64, BLOCK_SIZE_K=32, GROUP_SIZE_M=6
26         )
27     return c
28
29
30 cuda_generator = torch.Generator(device='cuda').manual_seed(42)
31 a_mat = torch.randn((128, 64), device='cuda', dtype=torch.float32, generator=cuda_generator)
32 b_mat = torch.randn((64, 128), device='cuda', dtype=torch.float32, generator=cuda_generator)
33
34 print(a_mat.is_contiguous())
35 print(b_mat.is_contiguous())
36
37 c_kernel = matmul(a_mat, b_mat, "our")
38 c_torch = torch.matmul(a_mat, b_mat)
39 c_openai_kernel = matmul(a_mat, b_mat, 'openai')
40
41 # print(c_kernel.is_contiguous())
42 # print(c_torch.is_contiguous())
43
44 print("Triton Matmul Result:\n", c_kernel)
45 print("Torch Matmul", c_torch)
46 torch.testing.assert_close(c_kernel, c_torch)
47 torch.testing.assert_close(c_kernel, c_openai_kernel)
```

Our implementation is identical to Pytorch, OpenAI Triton kernel. But it depend on several factors such as CUDA version, CUTLASS, CuDnn, Cublas, etc... to ensure the result is identical.

```
... True
True
Triton Matmul Result:
tensor([ [ 6.3183,  5.0860, -1.0062, ..., -6.1185,  3.5654, -8.4771],
        [ 17.1457,  2.1097, -3.0313, ...,  4.5661,  7.0699,  5.3764],
        [  3.9659,  1.5323,  3.2038, ...,  1.9161, -5.9991, -5.7718],
        ...,
        [  3.6513, -7.2427, -4.1788, ...,  7.9497, -3.0065, -0.1670],
        [  2.7735, -3.5068, -7.7250, ..., -2.5298, -7.4924, -13.9862],
        [-2.6971, -6.1373, -13.8907, ..., -2.7239, 13.7436,  0.7782]],
        device='cuda:0')
Torch Matmul tensor([ [ 6.3183,  5.0860, -1.0062, ..., -6.1185,  3.5654, -8.4771],
                     [ 17.1457,  2.1097, -3.0313, ...,  4.5661,  7.0699,  5.3764],
                     [  3.9659,  1.5323,  3.2038, ...,  1.9161, -5.9991, -5.7718],
                     ...,
                     [  3.6513, -7.2427, -4.1788, ...,  7.9497, -3.0065, -0.1670],
                     [  2.7735, -3.5068, -7.7250, ..., -2.5298, -7.4924, -13.9862],
                     [-2.6971, -6.1373, -13.8907, ..., -2.7239, 13.7436,  0.7782]],
                     device='cuda:0')
```

Effective Matrix Multiplication

This shows our implementation in Ada6000 Lovelace, the result is approximate only, if we set absolute difference is $< 1e-4$ then the result will fail.

```
)  
matmul_kernel[grid](  
    a, b, c,  
    M, N, K,  
    a.stride(0), a.stride(1),  
    b.stride(0), b.stride(1),  
    c.stride(0), c.stride(1),  
    BLOCK_SIZE_M=64, BLOCK_SIZE_N=64, BLOCK_SIZE_K=32,  
)  
return c  
  
cuda_generator = torch.Generator(device='cuda').manual_seed(42)  
a_mat = torch.randn((128, 64), device='cuda', dtype=torch.float32, generator=cuda_generator)  
b_mat = torch.randn((64, 128), device='cuda', dtype=torch.float32, generator=cuda_generator)  
  
c_kernel = matmul(a_mat, b_mat)  
c_torch = torch.matmul(a_mat, b_mat)  
  
print("Triton Matmul Result:\n", c_kernel)  
print("Torch Matmul", c_torch)  
torch.testing.assert_close(c_kernel, c_torch)
```

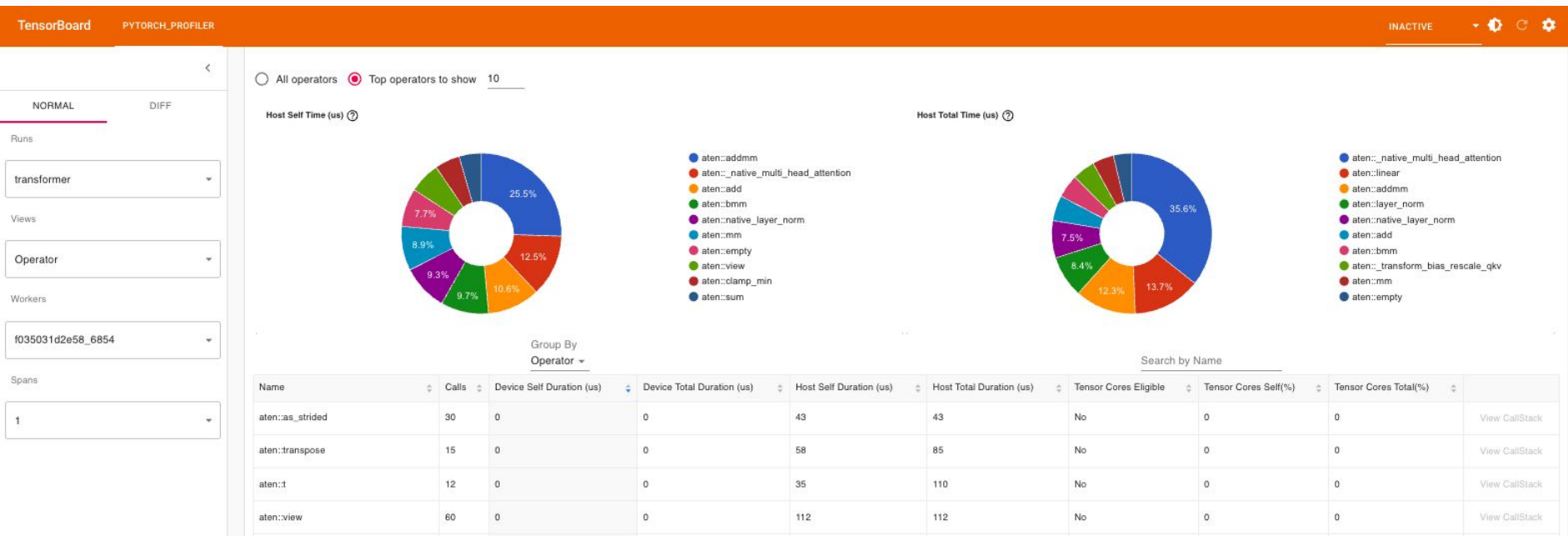
[5] 0.3s

```
... Triton Matmul Result:  
tensor([[ 6.3024,  5.0832, -1.0065, ..., -6.1137,  3.5634, -8.4728],  
        [ 17.1340,  2.1086, -3.0265, ...,  4.5636,  7.0664,  5.3736],  
        [  3.9629,  1.5338,  3.1992, ...,  1.9173, -5.9909, -5.7716],  
        ...,  
        [  3.6511, -7.2393, -4.1667, ...,  7.9455, -3.0020, -0.1634],  
        [  2.7727, -3.5086, -7.7233, ..., -2.5305, -7.4836, -13.9747],  
        [ -2.6941, -6.1312, -13.8773, ..., -2.7261, 13.7317,  0.7747]],  
        device='cuda:0')  
Torch Matmul tensor([[ 6.3103,  5.0860, -1.0062, ..., -6.1185,  3.5654, -8.4771],  
                    [ 17.1457,  2.1097, -3.0313, ...,  4.5661,  7.0699,  5.3764],  
                    [  3.9659,  1.5323,  3.2038, ...,  1.9161, -5.9991, -5.7718],  
                    ...,  
                    [  3.6513, -7.2427, -4.1708, ...,  7.9497, -3.0065, -0.1670],  
                    [  2.7735, -3.5068, -7.7250, ..., -2.5290, -7.4924, -13.9862],  
                    [ -2.6971, -6.1373, -13.8907, ..., -2.7239, 13.7436,  0.7782]],  
                    device='cuda:0')
```

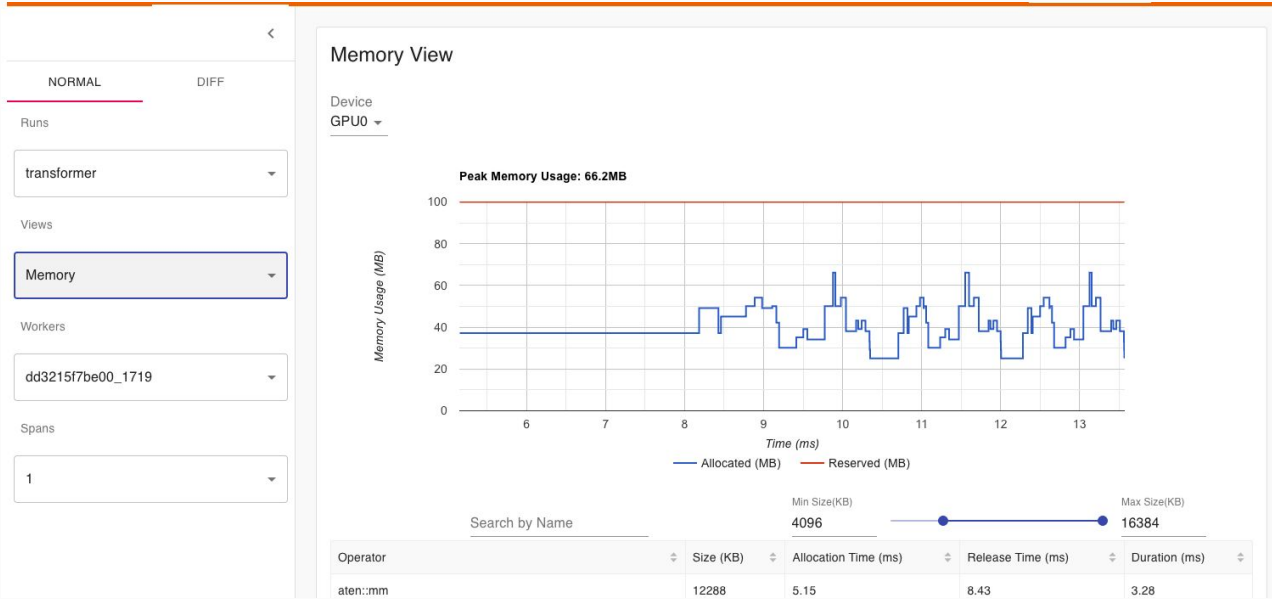
A decorative header with a blue geometric pattern of overlapping triangles and lines.

Torch Kernel Ecosystem

Torch Profiler



Torch Profiler



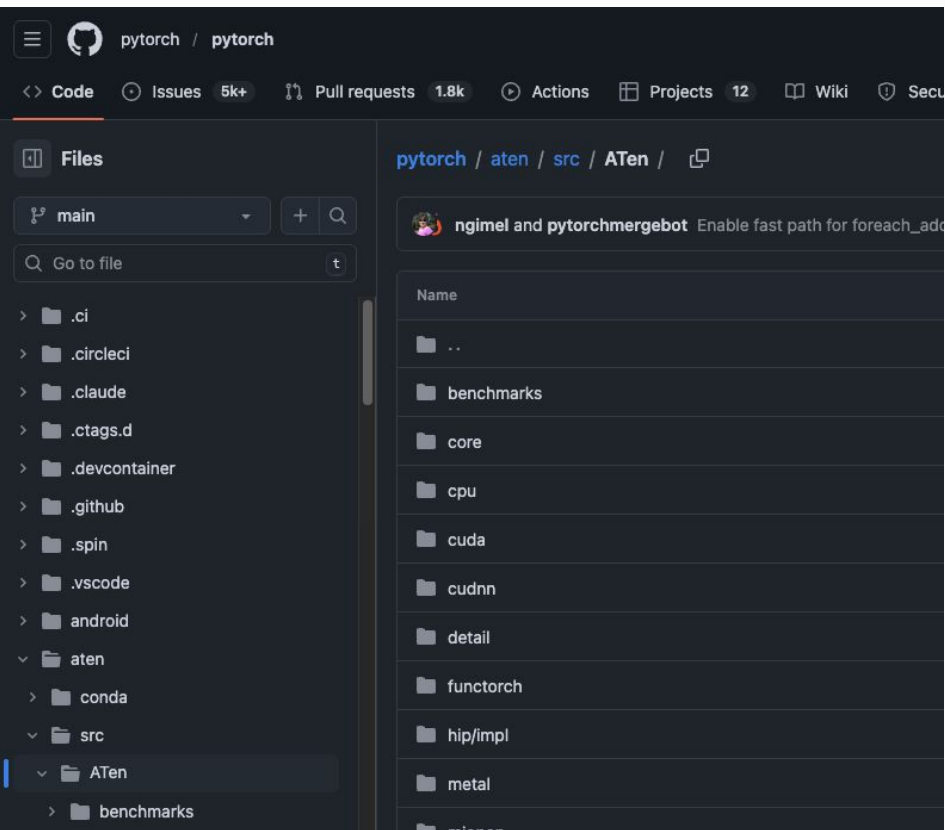
From Torch profiler, we can see for each operation conduct in our code, how many VRAM it consume, execution time, ... which give you insight where operation bottleneck your performance.

Torch Profiler

Operator	Size (KB)	Allocation Time (ms)	Release Time (ms)	Duration (ms)
aten::mm	12288	5.15	8.43	3.28
aten::empty_like (aten::empty)	5120	8.78	8.98	0.2
aten::bmm	8192	8.46	9.16	0.7
aten::_transform_bias_rescale_qkv (aten::empty)	12288	8.18	9.2	1.01
aten::add	5120	9.41	9.55	0.14

The **aten operators** is the CUDA Kernel written by Pytorch to customize and instruct how the calculation should be made

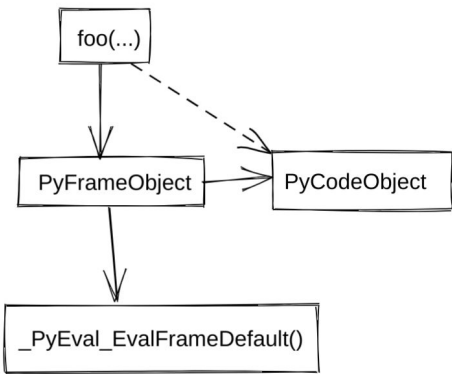
Torch Profiler



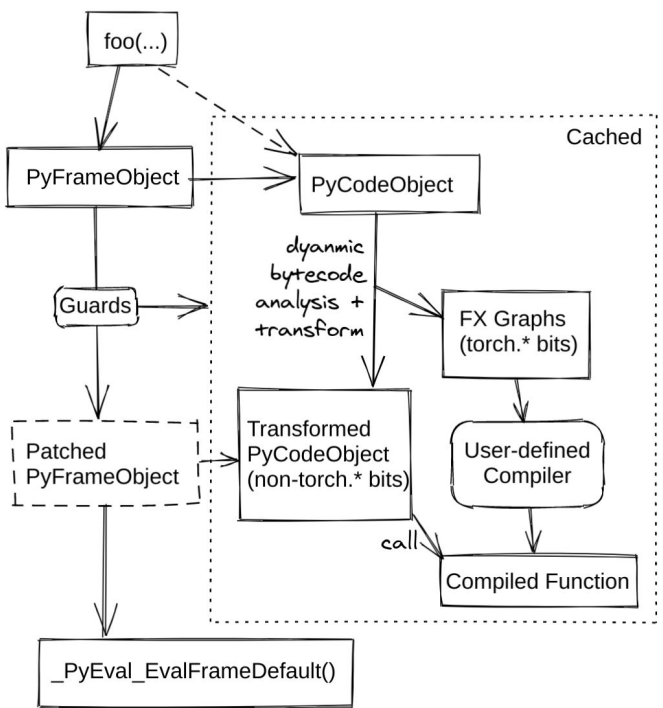
You can find the implementation of the aten module in the Pytorch Github, it written in C and CUDA to maximize hardware performance.

Torch Compile

Default Python Behavior

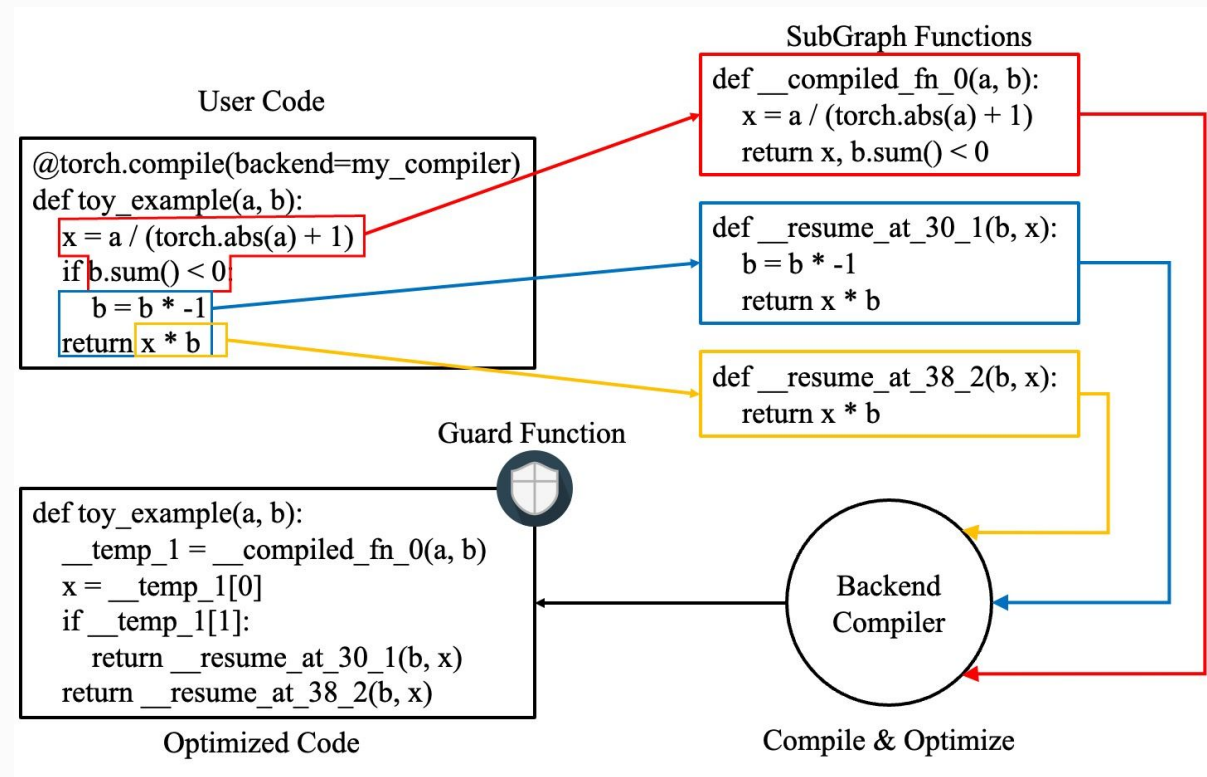


TorchDynamo Behavior



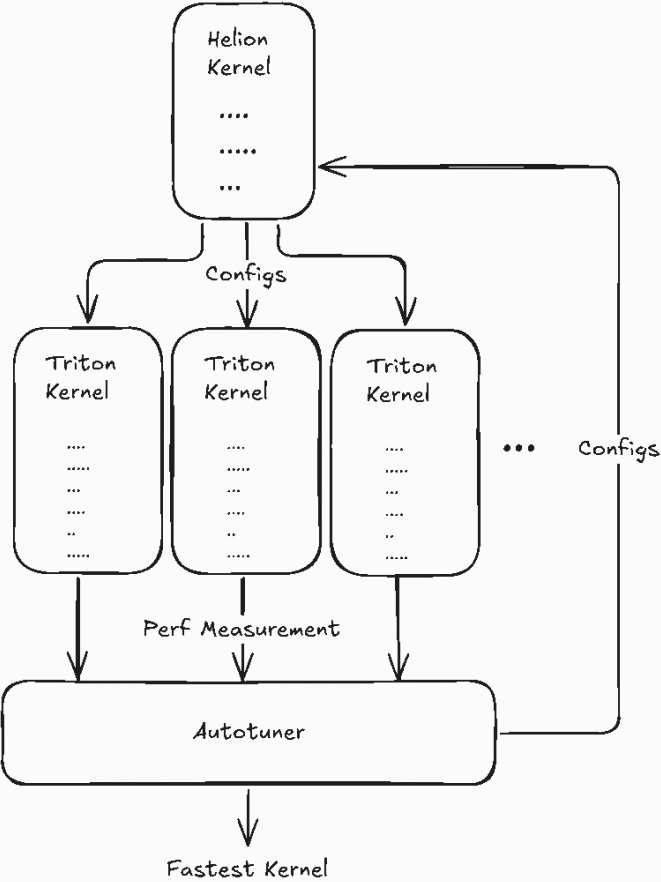
Torch compile is a JIT compiler enabled in Pytorch 2.0 to extend capability of compilation step of native Pytorch code into a graph to cache the compilation step.

Torch Compile



The example of torch compile where the compiler backend generate subgraph pytorch function.

Torch Helion



Torch Helion is a wrapper for Triton, its minimize boilerplate and leverage existing Pytorch knowledge

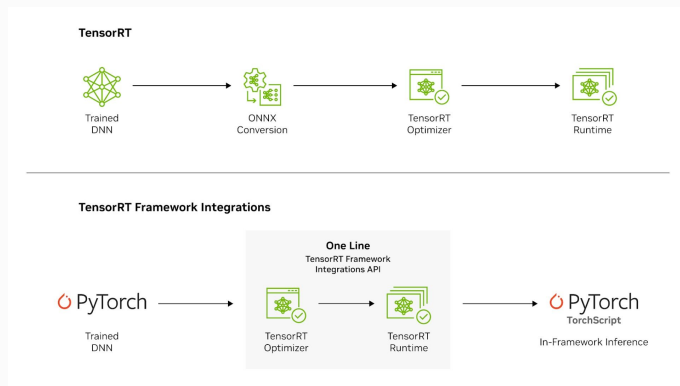
The top of the slide features a decorative header with various shades of blue and white geometric shapes, including triangles and lines.

Conclusion

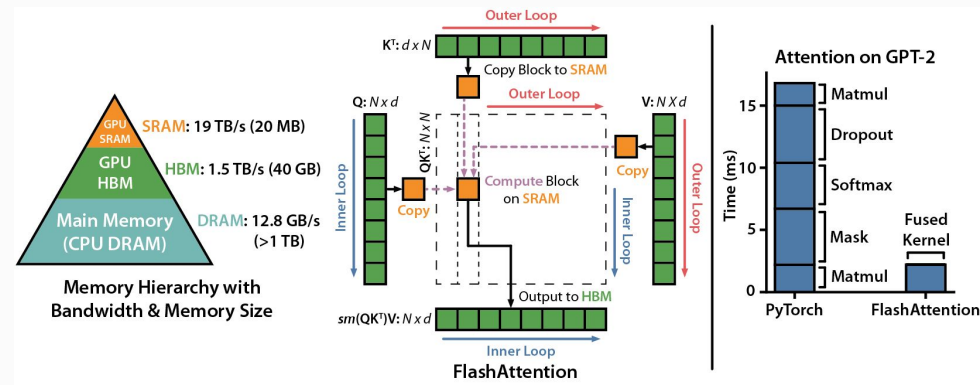
LLM Engine



LLM engine such as vLLM, SGLang or Triton Inference server write it own kernel to handle KV cache during inference.



FlashAttention



In the training phase, FlashAttention write custom GPU kernel to fuse operations in to one single op, reduce computation overhead due to memory transportation between SRAM and DRAM.

Conclusion

Writing your own GPU when

- You want to use open source model for both training and inference rather than proprietary
- You have team with prior knowledge to computer hardware
- Or you love how tensor is compute underneath the Torch wrapper (just like me)



noooooo you can't just scale up pure connectionist models on Internet data without inductive biases and modularization and expect them to learn real-world knowledge and grammar from form, or arithmetic and logical reasoning and causal inference—that's just memorization and superficial pattern-matching like Eliza, you need grounding in real-world communication with intent and social dynamics and multimodal robotic embodiment which can foster disentangled learning from guided exploration and self-directed goals expressed in Bayesian programs and probabilistic graphical models which are interpretable and pin down a unique semantics which can be debiased and expressed with uncertainty, and learned efficiently on tiny academic budgets, the cost only shows how this is a dead-end, we need to stop chasing SOTAs and model the complexity of the brain and consider the social context to decolonize AI's structural biases for Third World researchers...



haha gpus go bitterrrr

